Глубокое обучение с подкреплением

Модуль 4: градиент политики и REINFORCE

Курс Deep RL от Hugging Face

Полное учебное издание на основе исходных MDX-файлов

Исходный материал: курс Deep RL от Hugging Face.

Содержание

1 Введение **2**

2 Что такое методы, основанные на политике? **3**
- 2.1 Методы на основе ценности, на основе политики и Actor–Critic . . . 3
- 2.2 Различие между методами на основе политики и методами градиента политики . . . 5

3 Преимущества и недостатки методов градиента политики **5**
- 3.1 Преимущества . . . 5
- 3.2 Недостатки . . . 7

4 Более глубокое знакомство с методами градиента политики **8**
- 4.1 Общая картина . . . 8
- 4.2 Подробный разбор методов градиента политики . . . 9
- 4.3 Градиентный подъём и теорема о градиенте политики . . . 11
- 4.4 Алгоритм REINFORCE (Monte Carlo REINFORCE) . . . 13

5 (Необязательно) Теорема о градиенте политики **14**

6 Практическое занятие **16**
- 6.1 Модуль 4: напиши первый алгоритм глубокого RL на PyTorch — REINFORCE — и проверь его устойчивость . . . 17
- 6.2 Реализуем REINFORCE с нуля . . . 18
- 6.3 Первый агент: играем в CartPole-v1 . . . 21

7 Тест **36**
- 7.1 Ответы на тест и пояснения . . . 36

8 Заключение **37**

9 Глоссарий **38**

10 Дополнительная литература **38**
- 10.1 Введение в оптимизацию политики . . . 38
- 10.2 Градиент политики . . . 38
- 10.3 Реализация . . . 39

1 Введение

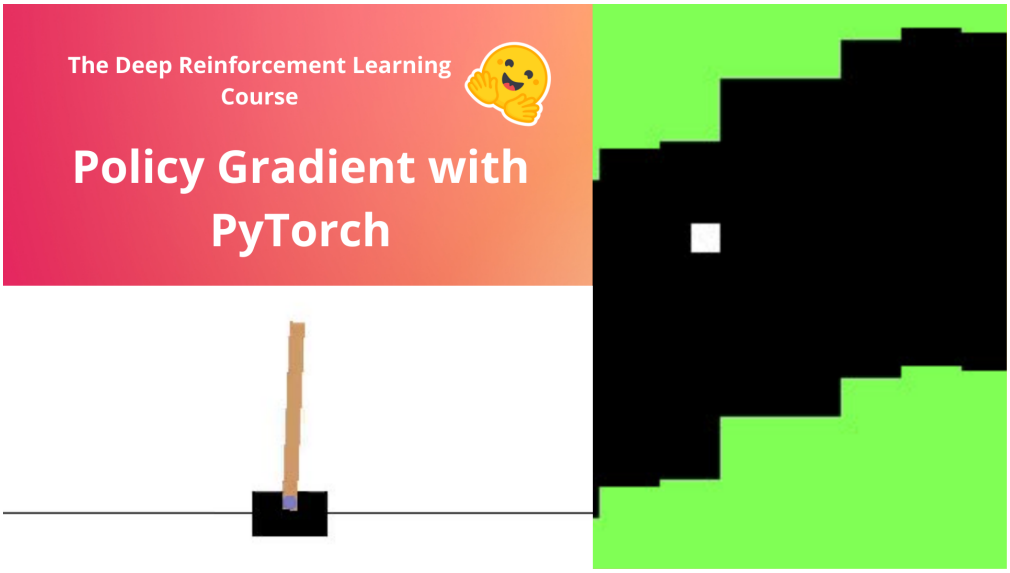


Рис. 1: Обложка модуля 4: градиент политики и REINFORCE [оригинал]

В предыдущем модуле мы изучили глубокое Q-обучение (Deep Q-Learning). В этом алгоритме глубокого обучения с подкреплением, основанном на оценке ценности, мы **использовали глубокую нейронную сеть для аппроксимации различных Q-значений каждого возможного действия в данном состоянии.**

С самого начала курса мы изучали только методы, основанные на оценке ценности, **где функция ценности служит промежуточным этапом на пути к оптимальной политике.**

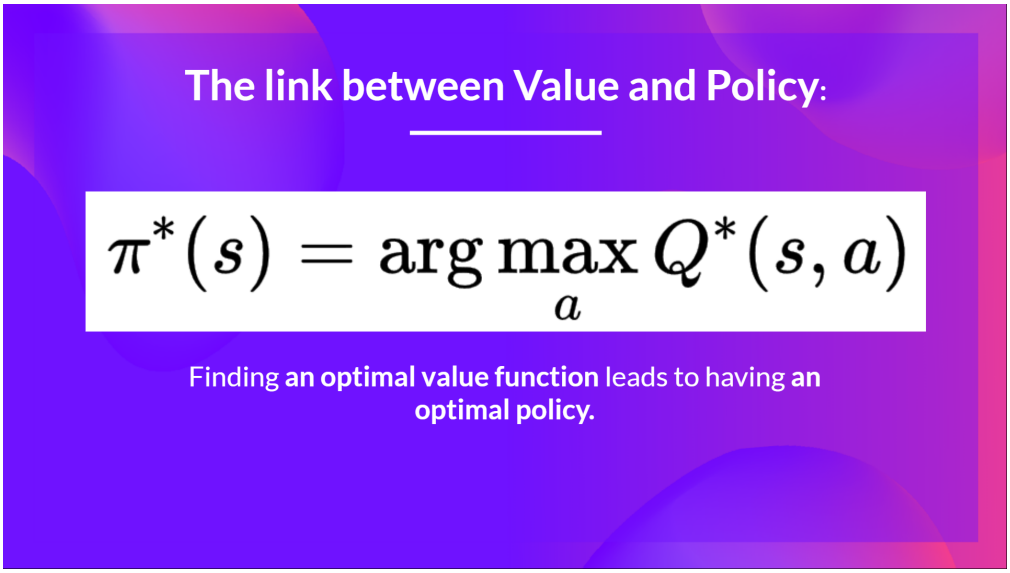


Рис. 2: Связь между оценками ценности действий и выбором политики [оригинал]

В методах, основанных на ценности, политика π **возникает на основе оценок ценности действий: по сути, это функция** (например, жадная политика), которая в заданном состоянии выбирает действие с максимальной ценностью.

В методах, основанных на политике, мы стремимся оптимизировать непосредственно политику **без промежуточного этапа обучения функции ценности.**

Сегодня мы **познакомимся с методами, основанными на политике, и изучим их подмножество — методы градиента политики (policy gradient)**. Затем с нуля реализуем на PyTorch первый алгоритм градиента политики — Monte Carlo **REINFORCE**. После этого

проверим устойчивость его работы в средах CartPole-v1 и PixelCopter.

Далее ты сможешь самостоятельно дорабатывать эту реализацию и применять её в более сложных средах.

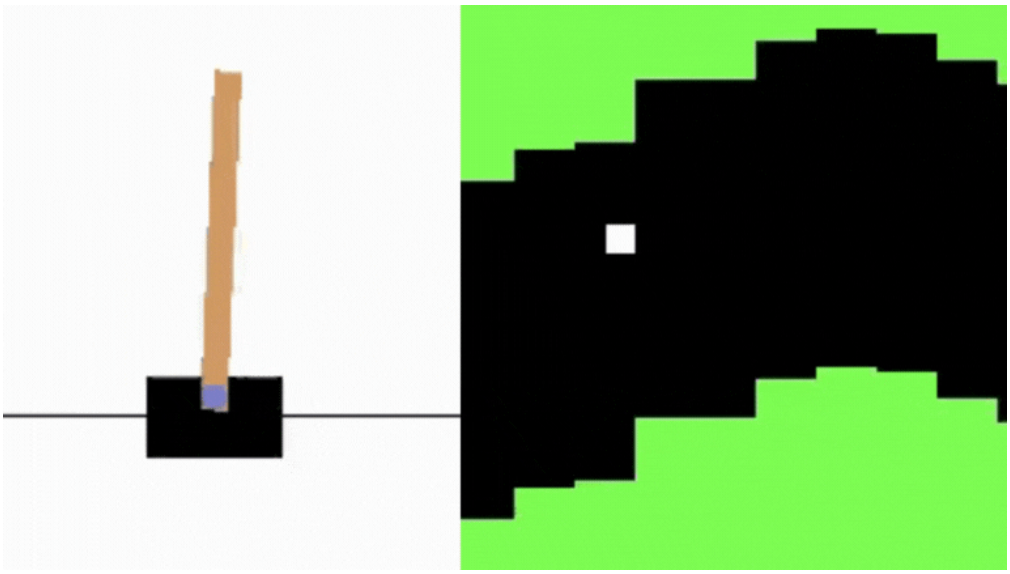


Рис. 3: Среды CartPole и PixelCopter (статичный кадр анимации) [оригинал]

Приступим!

2 Что такое методы, основанные на политике?

Основная задача обучения с подкреплением — **найти оптимальную политику π^*, максимизирующую ожидаемую суммарную награду**. В основе обучения с подкреплением лежит *гипотеза вознаграждения* (reward hypothesis): **любую цель можно выразить как максимизацию ожидаемой суммарной награды.**

Например, в футболе (агентов для которого ты будешь обучать через два модуля) цель — выиграть матч. На языке обучения с подкреплением её можно выразить как **максимизацию числа забитых сопернику голов** (когда мяч пересекает линию ворот) и **минимизацию числа голов, пропущенных в собственные ворота**.

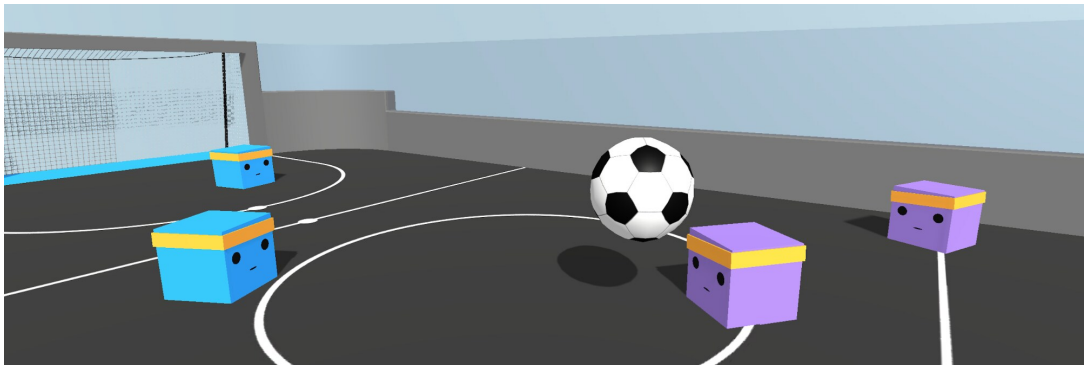


Рис. 4: Пример футбола для объяснения цели максимизации награды [оригинал]

2.1 Методы на основе ценности, на основе политики и Actor–Critic

В первом модуле мы рассмотрели два подхода к поиску (или, чаще, аппроксимации) этой оптимальной политики π^*.

- В *методах на основе ценности* мы обучаем функцию ценности.

  - Идея состоит в том, что оптимальная функция ценности позволяет получить оптимальную политику π^*.
  - Наша цель — **минимизировать ошибку между предсказанной и целевой ценностью**, чтобы аппроксимировать истинную функцию ценности действий.
  - Политика при этом существует, но задана неявно, поскольку **непосредственно строится по функции ценности**. Например, в Q-обучении мы использовали ε-жадную политику.

- Напротив, в *методах на основе политики* мы напрямую учимся аппроксимировать π^*, не обучая функцию ценности.
  - Основная идея — **параметризовать политику**. Например, нейронная сеть π_θ выдаёт распределение вероятностей действий (стохастическая политика).
  -

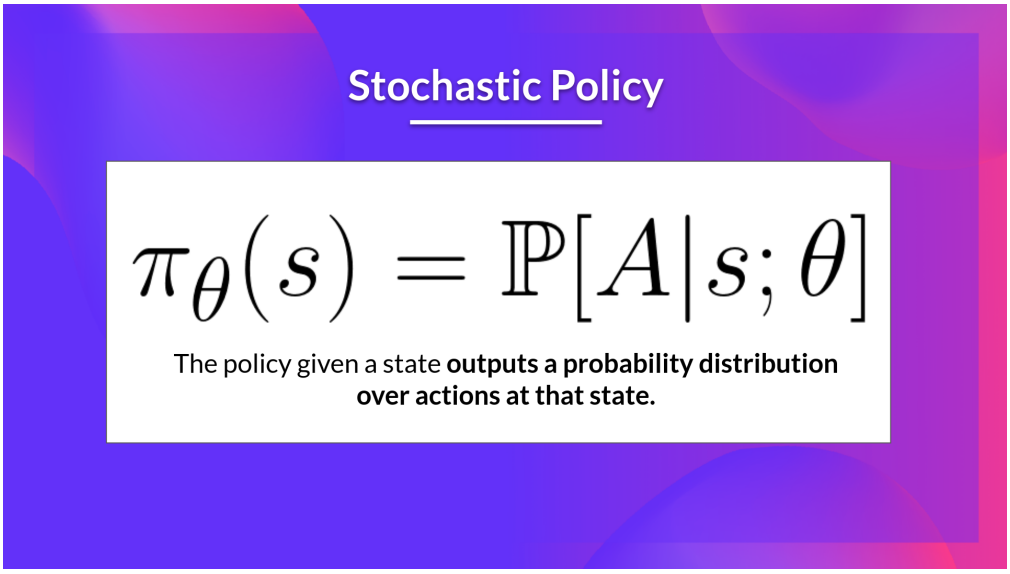


Рис. 5: Стохастическая политика выдаёт распределение вероятностей действий [оригинал]

- Следовательно, наша задача — **максимизировать качество параметризованной политики методом градиентного подъёма**.
- Для этого мы изменяем параметры θ, влияющие на распределение действий в каждом состоянии.

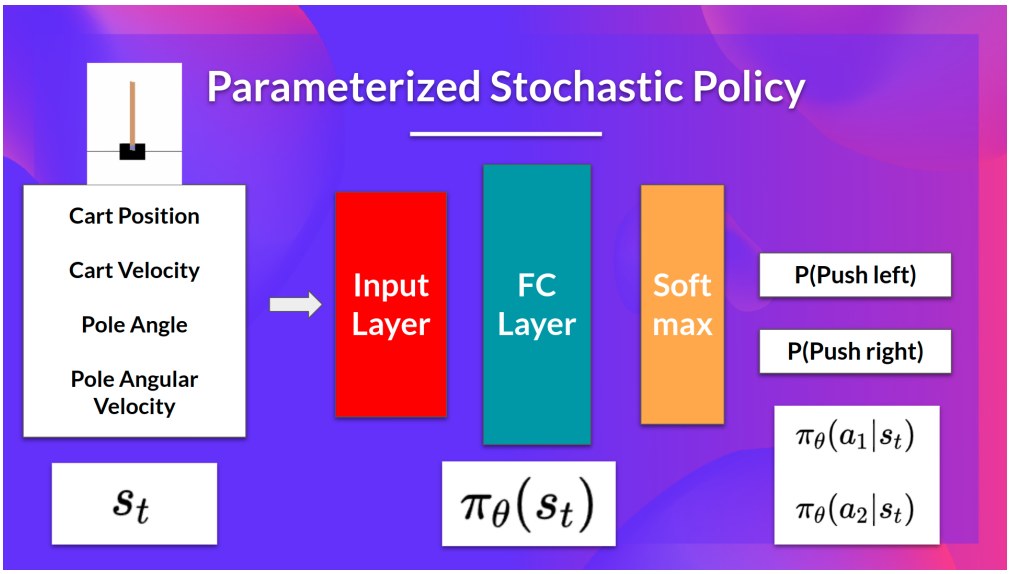


Рис. 6: Нейронная сеть политики: преобразование состояния в вероятности действий [оригинал]

- В следующем модуле мы изучим метод *Actor–Critic*, объединяющий подходы на основе ценности и на основе политики.

Таким образом, методы на основе политики позволяют непосредственно оптимизировать π_θ, чтобы получать распределение $\pi_\theta(a|s)$, обеспечивающее наибольшую суммарную отдачу. Для этого мы определяем целевую функцию $J(\theta)$, равную ожидаемой суммарной награде, и **ищем такое значение θ, при котором она максимальна**.

2.2 Различие между методами на основе политики и методами градиента политики

Методы градиента политики, которым посвящён этот модуль, представляют собой подкласс методов на основе политики. Для таких методов оптимизация чаще всего происходит *on-policy*, поскольку на каждом шаге обновления мы используем только данные (траектории), собранные **самой последней версией политики π_θ**.

Разница между этими подходами **заключается в способе оптимизации параметров θ**:

- В *методах на основе политики* мы непосредственно ищем оптимальную политику. Параметры θ можно оптимизировать **косвенно**, максимизируя локальную аппроксимацию целевой функции с помощью восхождения на холм (hill climbing), имитации отжига или эволюционных стратегий.
- В *методах градиента политики*, как подклассе методов на основе политики, мы также непосредственно ищем оптимальную политику. Но параметры θ оптимизируются **напрямую** посредством градиентного подъёма целевой функции $J(\theta)$.

Прежде чем подробно разбирать работу методов градиента политики (целевую функцию, теорему о градиенте политики, градиентный подъём и т. д.), рассмотрим их преимущества и недостатки.

3 Преимущества и недостатки методов градиента политики

На этом этапе можно спросить: «Но ведь глубокое Q-обучение отлично работает! Зачем использовать градиент политики?» Чтобы ответить, рассмотрим **преимущества и недостатки методов градиента политики**.

3.1 Преимущества

По сравнению с методами на основе ценности у них есть несколько достоинств. Рассмотрим некоторые из них.

Простота интеграции

Мы можем непосредственно оценивать политику, не сохраняя дополнительных данных (ценностей действий).

Методы градиента политики способны обучать стохастическую политику

Методы градиента политики могут **обучать стохастическую политику, тогда как функции ценности сами по себе такой возможности не дают**.

Отсюда следуют два последствия:

1. Нам **не требуется вручную задавать компромисс между исследованием среды и использованием найденного (exploration/exploitation)**. Поскольку на выходе получаем распределение вероятностей действий, агент исследует **пространство состояний, не повторяя постоянно одну и ту же траекторию.**

2. Мы также избавляемся от проблемы **неразличимости состояний по наблюдениям** (perceptual aliasing). Она возникает, когда два состояния выглядят одинаковыми (или действительно совпадают с точки зрения наблюдений), но требуют разных действий.

Рассмотрим пример: у нас есть умный робот-пылесос, который должен собирать пыль и при этом не причинять вреда хомякам.

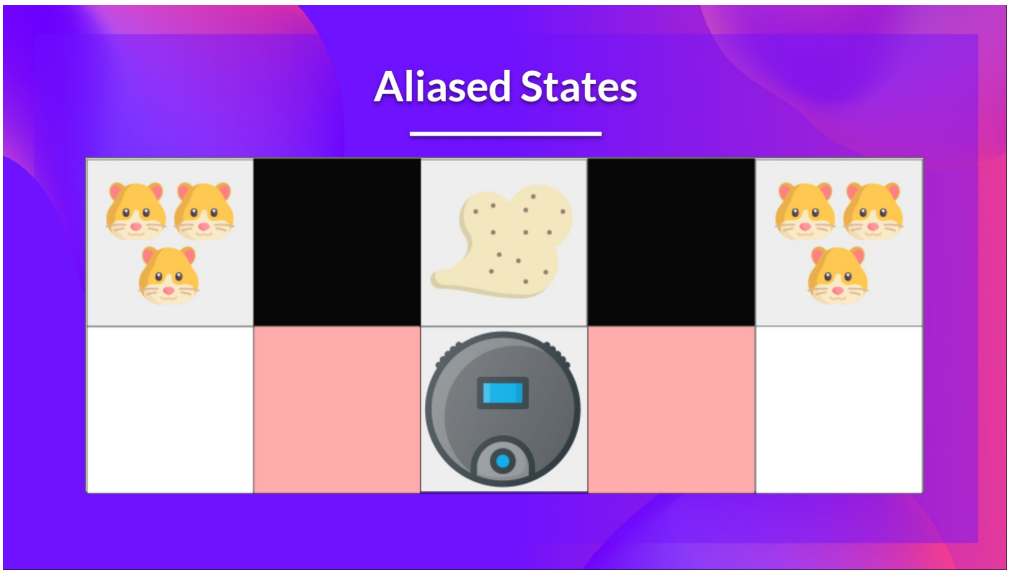


Рис. 7: Стохастические политики: первая иллюстрация с роботом-пылесосом [оригинал]

Наш робот-пылесос умеет определять только расположение стен.

Проблема в том, что **два отмеченных красным состояния неразличимы для агента: в обоих он воспринимает стену сверху и снизу**.

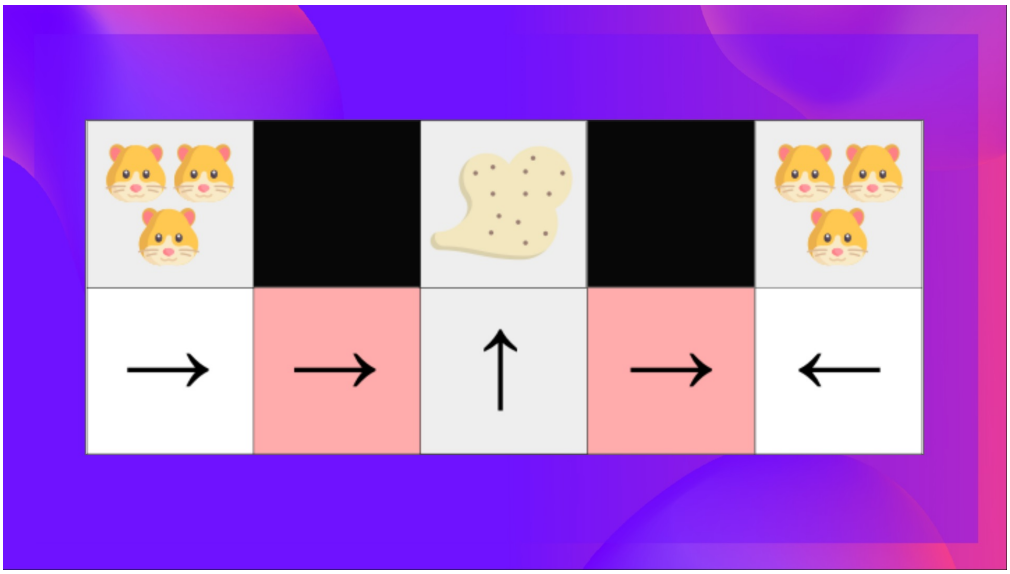


Рис. 8: Стохастические политики: вторая иллюстрация [оригинал]

При детерминированной политике агент в красном состоянии либо всегда идёт вправо, либо всегда влево. **В обоих случаях он застрянет и так и не доберётся до пыли**.

Если использовать алгоритм обучения с подкреплением на основе ценности, то мы получим **почти детерминированную политику** («ε-жадную стратегию»). В результате агент может **очень долго искать пыль**.

Напротив, оптимальная стохастическая политика **случайным образом выбирает движение влево или вправо в красных состояниях**. Благодаря этому **агент не застрянет и с**

высокой вероятностью достигнет цели.

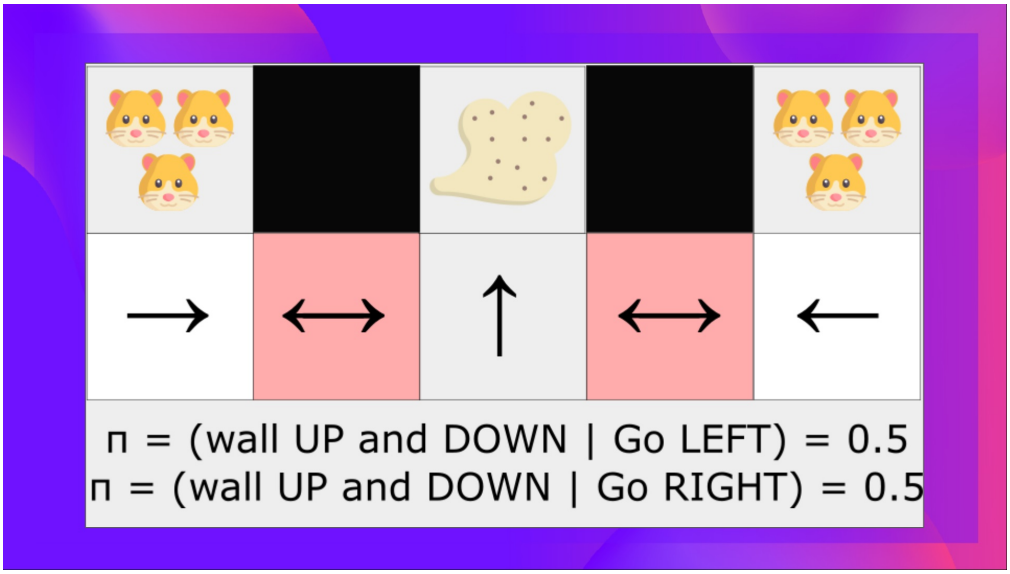


Рис. 9: Стохастические политики: третья иллюстрация [оригинал]

Методы градиента политики эффективнее в пространствах действий высокой размерности и с непрерывными действиями

Проблема глубокого Q-обучения в том, что для каждого состояния на каждом шаге алгоритм **предсказывает оценку (максимальную ожидаемую будущую награду) для каждого возможного действия**.

Но что, если возможных действий бесконечно много?

Например, у беспилотного автомобиля в каждом состоянии может быть (практически) бесконечное множество действий: повернуть руль на 15°, 17,2°, 19,4°, посигналить и т. д. **Нам пришлось бы получать Q-значение для каждого возможного действия**! К тому же **поиск максимума среди непрерывного множества действий сам по себе представляет задачу оптимизации**!

Вместо этого методы градиента политики выдают **распределение вероятностей по действиям**.

Методы градиента политики обладают более благоприятными свойствами сходимости

В методах на основе ценности мы используем достаточно жёсткий оператор для **изменения функции ценности: выбираем максимум среди Q-оценок**. Поэтому вероятности выбора действий могут резко меняться даже при сколь угодно малом изменении оценок, если в результате максимальное значение получает другое действие.

Например, если во время обучения лучшим действием было движение влево (Q-значение 0,22), а уже после следующего шага — вправо (Q-значение для него стало 0,23), то политика резко изменится: теперь агент почти всегда будет двигаться вправо, а не влево.

Напротив, в методах градиента политики предпочтения стохастической политики (вероятности выбора действий) **изменяются плавно с течением времени**.

3.2 Недостатки

Разумеется, методы градиента политики имеют и свои недостатки:

- **Методы градиента политики нередко сходятся к локальному максимуму вместо глобального оптимума.**

- Градиентный подъём происходит медленно, **шаг за шагом: обучение может занимать больше времени (неэффективность по выборке)**.
- Оценки градиента политики могут иметь высокую дисперсию. В модуле об Actor–Critic мы разберём причины и способы решения этой проблемы.

Если хочешь подробнее узнать о преимуществах и недостатках методов градиента политики, посмотри это видео.

4 Более глубокое знакомство с методами градиента политики

4.1 Общая картина

Мы только что узнали, что методы градиента политики ищут параметры θ, **максимизирующие ожидаемую отдачу**.

Основная идея: у нас есть *параметризованная стохастическая политика*. В нашем случае нейронная сеть выдаёт распределение вероятностей действий. Вероятность выбора каждого действия также называют *предпочтением действия* (action preference).

Рассмотрим среду CartPole-v1:

- На вход подаётся состояние.
- На выходе получаем распределение вероятностей возможных действий в этом состоянии.

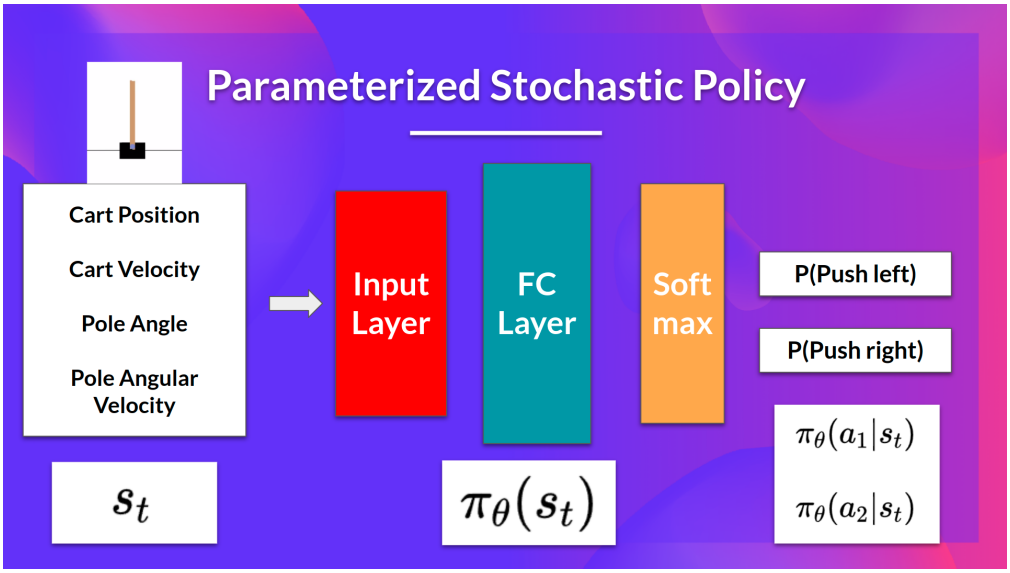


Рис. 10: Нейронная сеть политики: преобразование состояния в вероятности действий [оригинал]

В методах градиента политики мы хотим **управлять распределением вероятностей действий**, настраивая политику так, чтобы **хорошие действия (повышающие отдачу) в дальнейшем выбирались чаще**. После каждого взаимодействия агента со средой мы корректируем параметры, увеличивая вероятность выбора удачных действий в будущем.

Но **как оптимизировать веса с помощью ожидаемой отдачи**?

Идея заключается в том, чтобы **позволить агенту взаимодействовать со средой в течение эпизода**. Если эпизод завершился успехом, считаем, что выполненные действия были хорошими и впоследствии их стоит выбирать чаще, поскольку они привели к победе.

Следовательно, для каждой пары «состояние–действие» мы хотим увеличить $P(a|s)$ — вероятность выбрать данное действие в данном состоянии. А если эпизод завершился неудачей, эту вероятность нужно уменьшить.

Упрощённая схема алгоритма градиента политики выглядит так:

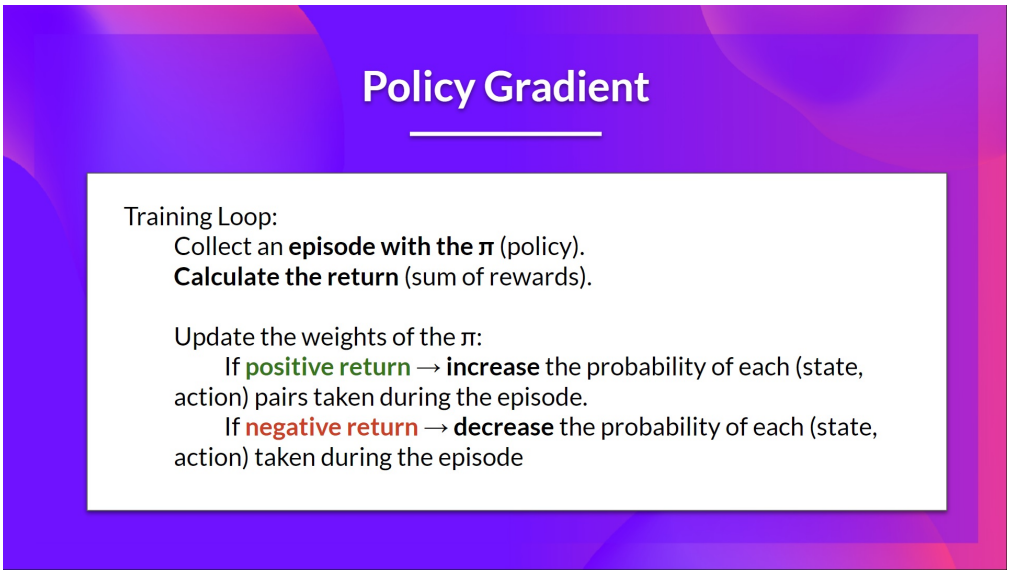


Рис. 11: Общая схема обновления параметров в методе градиента политики [оригинал]

Теперь, когда общая картина понятна, углубимся в устройство методов градиента политики.

4.2 Подробный разбор методов градиента политики

У нас есть стохастическая политика π с параметрами θ. Получая состояние, эта политика **выдаёт распределение вероятностей действий**.

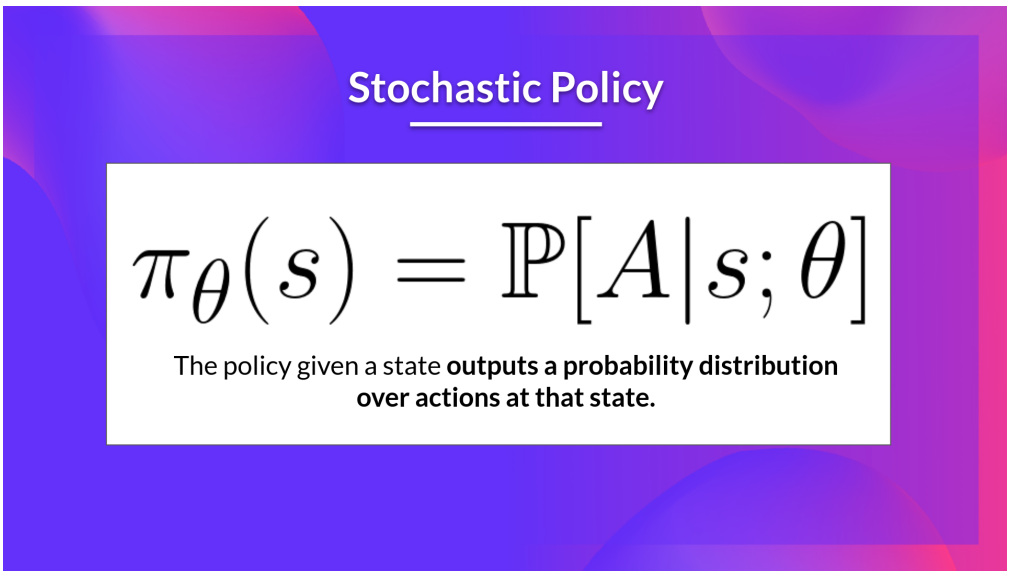


Рис. 12: Стохастическая политика выдаёт распределение вероятностей действий [оригинал]

Здесь $\pi_\theta(a_t|s_t)$ — вероятность того, что агент, следуя заданной политике, выберет действие a_t в состоянии s_t.

Но как определить, хороша ли наша политика? Нужна количественная мера её качества. Для этого введём функцию оценки, или целевую функцию, $J(\theta)$.

Целевая функция

Целевая функция характеризует **качество работы агента** с учётом траектории (последовательности состояний и действий без учёта наград, в отличие от полного описания эпизода) и возвращает *ожидаемую суммарную награду*.

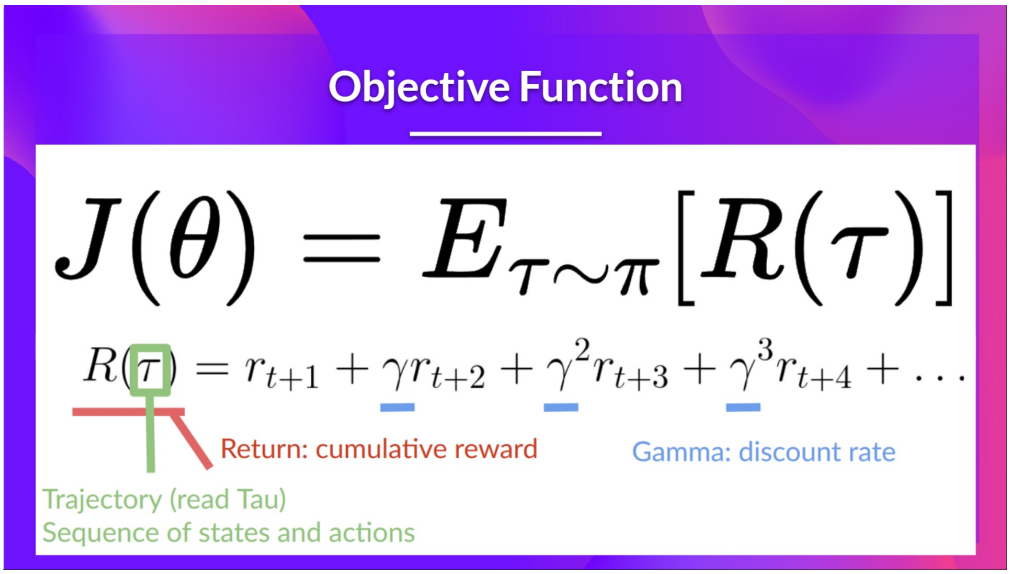


Рис. 13: Целевая функция градиента политики [оригинал]

Разберём эту формулу подробнее:

- *Ожидаемая отдача* (или ожидаемая суммарная награда) представляет собой взвешенное среднее всех возможных значений отдачи $R(\tau)$, причём веса определяются вероятностями $P(\tau;\theta)$.

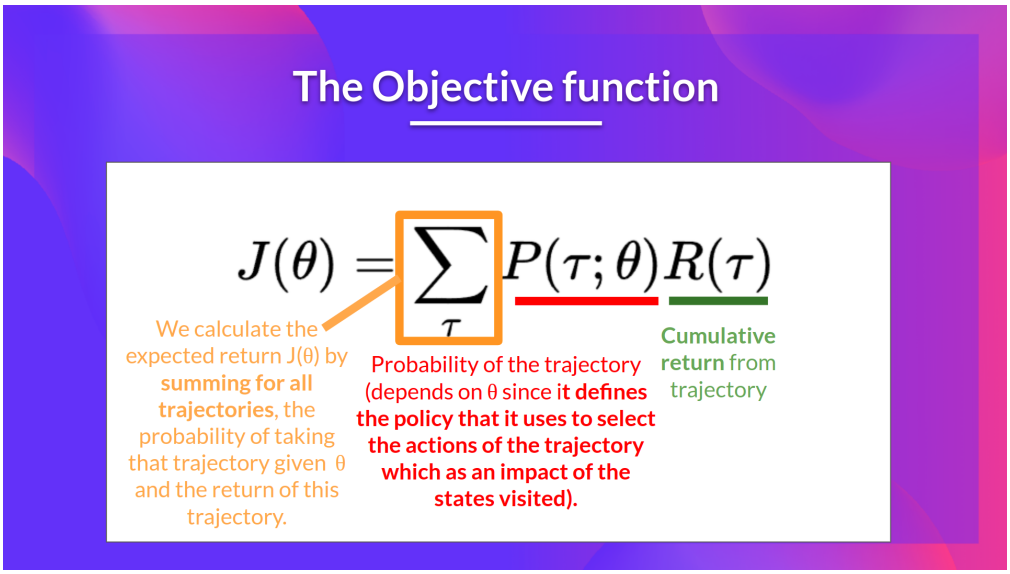


Рис. 14: Ожидаемая отдача как целевая функция оптимизации [оригинал]

- $R(\tau)$ — отдача от произвольной траектории. Чтобы использовать её для вычисления ожидаемой отдачи, нужно умножить её на вероятность соответствующей траектории.
- $P(\tau;\theta)$ — вероятность конкретной траектории τ. Она зависит от θ, поскольку эти параметры определяют политику выбора действий, а значит, влияют и на посещаемые состояния.

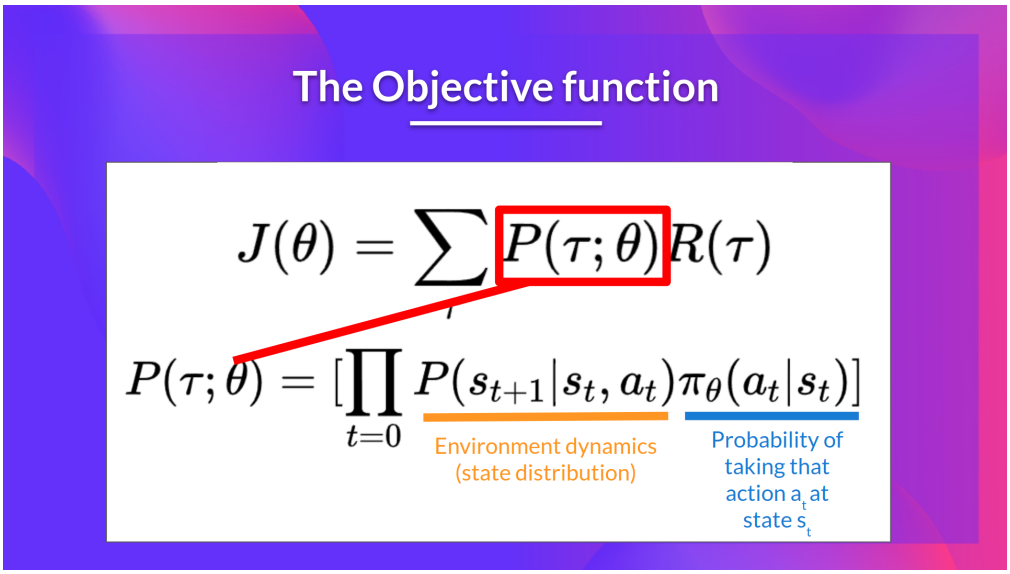


Рис. 15: Вероятность траектории при параметризованной политике [оригинал]

- $J(\theta)$ — ожидаемая отдача: для её вычисления суммируем по всем траекториям произведения вероятности каждой траектории при заданных θ на отдачу этой траектории.

Таким образом, наша цель — максимизировать ожидаемую суммарную награду, найдя параметры θ, формирующие наилучшие распределения вероятностей действий:

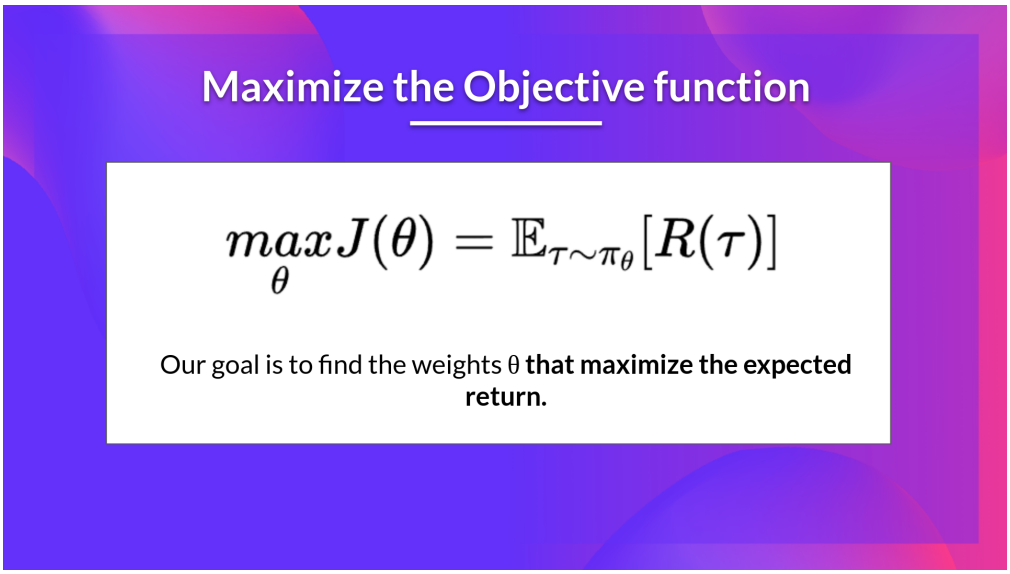


Рис. 16: Максимизация целевой функции по параметрам политики [оригинал]

4.3 Градиентный подъём и теорема о градиенте политики

Градиент политики — задача оптимизации: мы хотим найти значения θ, при которых целевая функция $J(\theta)$ максимальна. Поэтому используем **градиентный подъём** (gradient ascent). Это противоположность *градиентному спуску*: градиент задаёт направление наиболее быстрого увеличения $J(\theta)$.

Если нужно вспомнить различие между градиентным спуском и подъёмом, прочитай эту статью и это объяснение.

Шаг обновления при градиентном подъёме записывается так:

$\theta \leftarrow \theta + \alpha * \nabla_\theta J(\theta)$

Мы можем многократно применять это обновление, рассчитывая на сходимость θ к значению, максимизирующему $J(\theta)$.

Однако при вычислении производной $J(\theta)$ возникают две проблемы:

1. Мы не можем вычислить точный градиент целевой функции: для этого нужно рассчитать вероятности всех возможных траекторий, что вычислительно чрезвычайно дорого. Поэтому мы хотим **оценивать градиент по выборке (собирая некоторое число траекторий)**.
2. Есть и другая проблема, которую я объясняю в следующем необязательном разделе. Для дифференцирования целевой функции пришлось бы дифференцировать распределение состояний, связанное с динамикой марковского процесса принятия решений (MDP). Эта динамика определяется средой: она задаёт вероятность перехода в следующее состояние при известном текущем состоянии и действии агента. Но мы можем не знать эту динамику и потому не можем её дифференцировать.

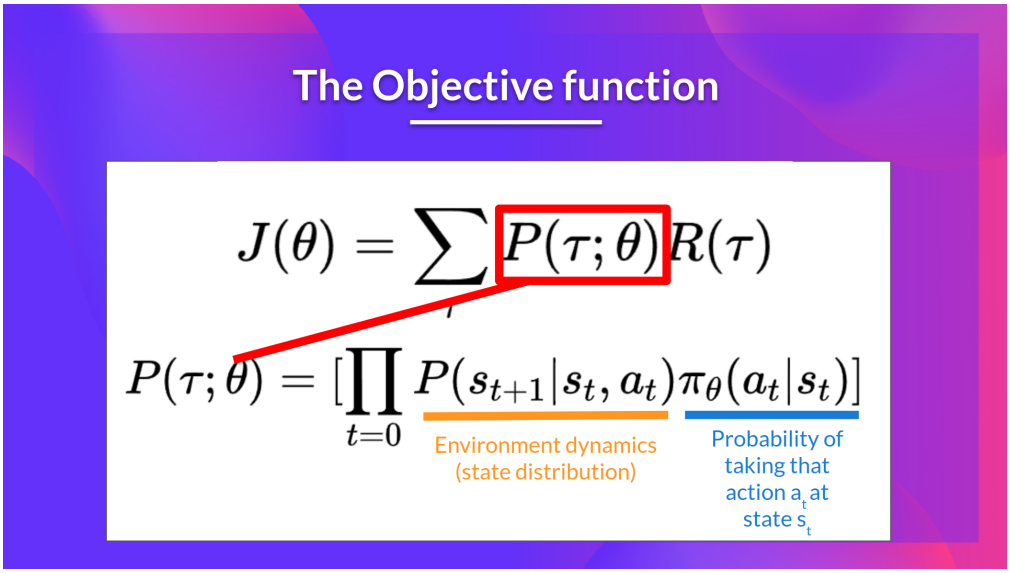


Рис. 17: Вероятность траектории при параметризованной политике [оригинал]

К счастью, существует решение — *теорема о градиенте политики* (Policy Gradient Theorem). Она позволяет преобразовать выражение для градиента целевой функции так, чтобы не требовалось дифференцировать распределение состояний.

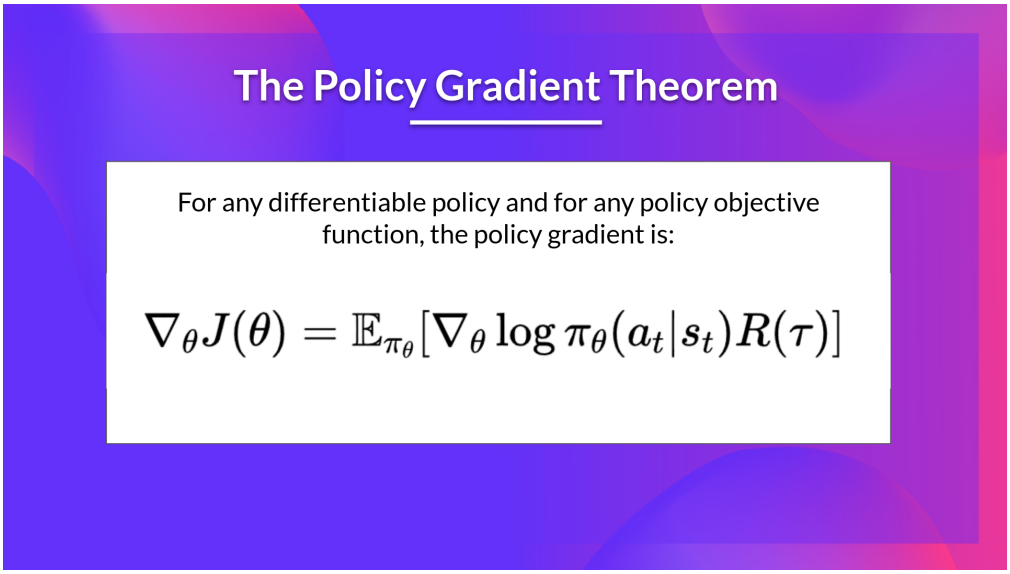


Рис. 18: Теорема о градиенте политики [оригинал]

Если хочешь понять, как выводится эта формула оценки градиента, перейди к следующему (необязательному) разделу.

4.4 Алгоритм REINFORCE (Monte Carlo REINFORCE)

Алгоритм REINFORCE, также называемый методом градиента политики Монте-Карло, — это алгоритм градиента политики, который **использует оценку отдачи за целый эпизод для обновления параметров политики** θ:

В цикле:

- Используем политику π_θ, чтобы собрать эпизод τ.
- По этому эпизоду оцениваем градиент $\hat{g} = \nabla_\theta J(\theta)$.

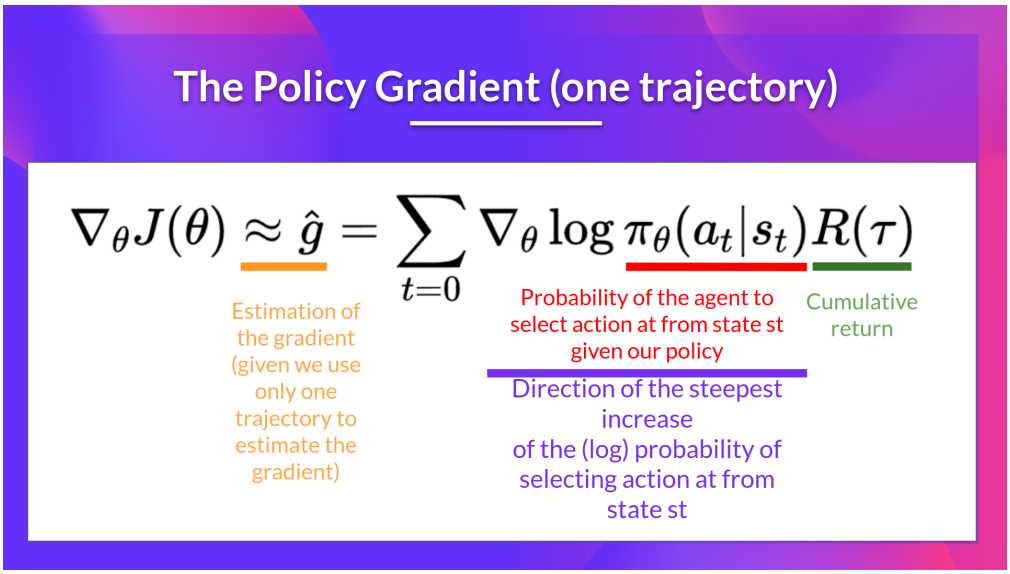


Рис. 19: Обновление REINFORCE по одной выбранной траектории [оригинал]

- Обновляем веса политики: $\theta \leftarrow \theta + \alpha\hat{g}$.

Это обновление можно интерпретировать следующим образом:

- $\nabla_\theta log \pi_\theta(a_t|s_t)$ — направление **наиболее быстрого увеличения логарифма вероятности** выбора действия a_t в состоянии s_t. Этот градиент показывает, **как нужно изменить веса политики**, чтобы увеличить или уменьшить логарифм вероятности выбора a_t в s_t.
- $R(\tau)$ — функция оценки качества:
  - Если отдача велика, она будет **повышать вероятности** соответствующих пар (состояние, действие).
  - Если отдача мала, она будет **понижать вероятности** соответствующих пар (состояние, действие).

Для оценки градиента можно также **собирать несколько эпизодов (траекторий)**:

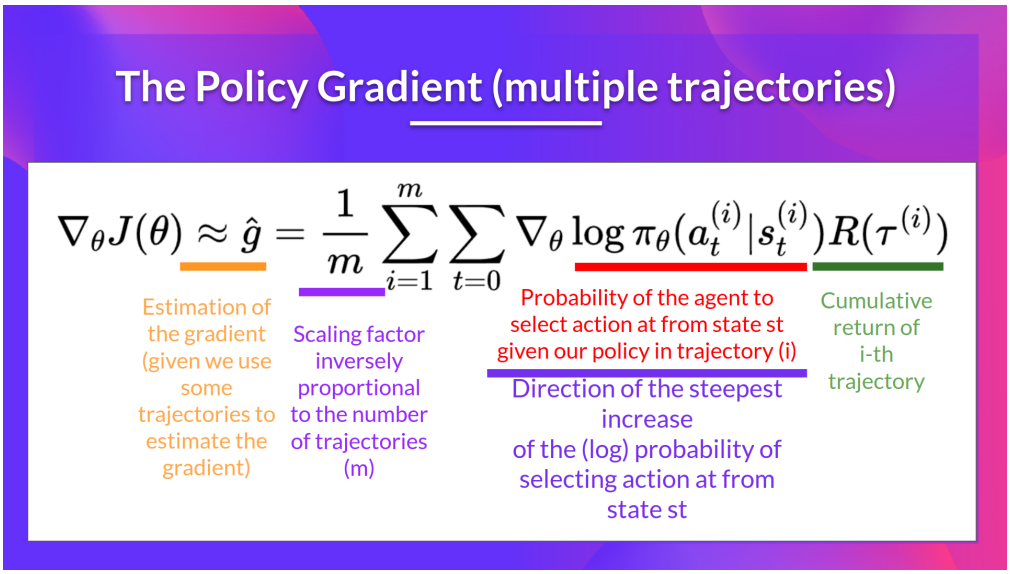


Рис. 20: Обновление REINFORCE с усреднением по нескольким траекториям [оригинал]

5 (Необязательно) Теорема о градиенте политики

В этом необязательном разделе мы **разберём, как дифференцировать целевую функцию для приближённого вычисления градиента политики**.

Для начала вспомним наши формулы:

1. Целевая функция.

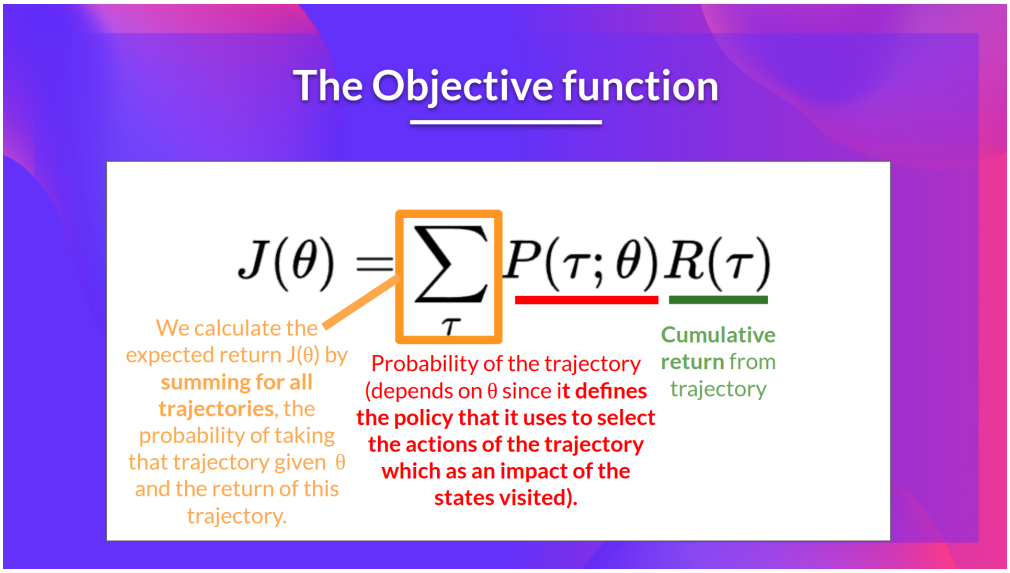


Рис. 21: Ожидаемая отдача как целевая функция оптимизации [оригинал]

2. Вероятность траектории (при условии, что действия выбираются политикой π_θ):

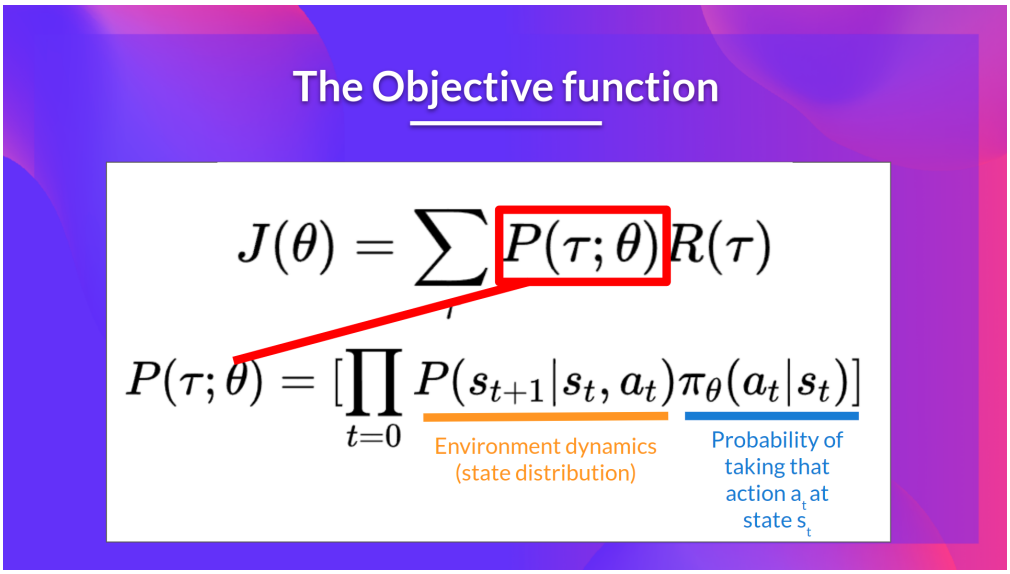


Рис. 22: Вероятность траектории при параметризованной политике [оригинал]

Итак, имеем:

$\nabla_\theta J(\theta) = \nabla_\theta \sum_\tau P(\tau;\theta)R(\tau)$

Градиент суммы можно представить как сумму градиентов:

$= \sum_\tau \nabla_\theta (P(\tau;\theta)R(\tau)) = \sum_\tau \nabla_\theta P(\tau;\theta)R(\tau)$, поскольку $R(\tau)$ не зависит от θ.

Затем умножим каждый член суммы на $\frac{P(\tau;\theta)}{P(\tau;\theta)}$ (это допустимо, поскольку отношение равно 1).

$= \sum_\tau \frac{P(\tau;\theta)}{P(\tau;\theta)} \nabla_\theta P(\tau;\theta)R(\tau)$

Далее упростим выражение, так как $\frac{P(\tau;\theta)}{P(\tau;\theta)} \nabla_\theta P(\tau;\theta) = P(\tau;\theta)\frac{\nabla_\theta P(\tau;\theta)}{P(\tau;\theta)}$.

Поэтому сумму можно переписать следующим образом:

$P(\tau;\theta)\frac{\nabla_\theta P(\tau;\theta)}{P(\tau;\theta)} = \sum_\tau P(\tau;\theta)\frac{\nabla_\theta P(\tau;\theta)}{P(\tau;\theta)} R(\tau)$

Теперь воспользуемся *трюком с производной логарифма* (derivative log trick), который также называют *трюком отношения правдоподобий* (likelihood ratio trick) или *трюком REINFORCE*. Это простое правило анализа: $\nabla_x log f(x) = \frac{\nabla_x f(x)}{f(x)}$.

Поскольку в выражении присутствует $\frac{\nabla_\theta P(\tau;\theta)}{P(\tau;\theta)}$, заменим его на $\nabla_\theta log P(\tau|\theta)$.

Получаем градиент политики через правдоподобие:

$\nabla_\theta J(\theta) = \sum_\tau P(\tau;\theta)\nabla_\theta log P(\tau;\theta)R(\tau)$

Благодаря новой формуле градиент можно оценивать по выборке траекторий (или, что то же самое, выборочно аппроксимировать градиент через отношение правдоподобий).

$\nabla_\theta J(\theta) = \frac{1}{m}\sum_{i=1}^{m} \nabla_\theta log P(\tau^{(i)};\theta)R(\tau^{(i)})$, где каждая $\tau^{(i)}$ — выбранная из данных траектория.

Но математические преобразования ещё не закончены: требуется упростить $\nabla_\theta log P(\tau|\theta)$.

Мы знаем, что:

$\nabla_\theta log P(\tau^{(i)};\theta) = \nabla_\theta log[\mu(s_0)\prod_{t=0}^{H} P(s_{t+1}^{(i)}|s_t^{(i)}, a_t^{(i)})\pi_\theta(a_t^{(i)}|s_t^{(i)})]$

Здесь $\mu(s_0)$ — распределение начальных состояний, а $P(s_{t+1}^{(i)}|s_t^{(i)}, a_t^{(i)})$ — динамика переходов MDP.

Логарифм произведения равен сумме логарифмов:

$$\nabla_\theta log P(\tau^{(i)};\theta) = \nabla_\theta \left[log\mu(s_0) + \sum_{t=0}^{H} log P(s_{t+1}^{(i)}|s_t^{(i)}a_t^{(i)}) + \sum_{t=0}^{H} log\pi_\theta(a_t^{(i)}|s_t^{(i)}) \right]$$

Кроме того, градиент суммы равен сумме градиентов:

$\nabla_\theta logP(\tau^{(i)};\theta) = \nabla_\theta log\mu(s_0) + \nabla_\theta \sum_{t=0}^{H} logP(s_{t+1}^{(i)}|s_t^{(i)}a_t^{(i)}) + \nabla_\theta \sum_{t=0}^{H} log\pi_\theta(a_t^{(i)}|s_t^{(i)})$

Поскольку ни распределение начальных состояний, ни динамика переходов MDP не зависят от θ, производные обоих слагаемых равны нулю. Поэтому их можно убрать:

Поскольку $\nabla_\theta \sum_{t=0}^{H} logP(s_{t+1}^{(i)}|s_t^{(i)}a_t^{(i)}) = 0$ и $\nabla_\theta \mu(s_0) = 0$.

$\nabla_\theta logP(\tau^{(i)};\theta) = \nabla_\theta \sum_{t=0}^{H} log\pi_\theta(a_t^{(i)}|s_t^{(i)})$

Градиент суммы вновь перепишем как сумму градиентов:

$\nabla_\theta logP(\tau^{(i)};\theta) = \sum_{t=0}^{H} \nabla_\theta log\pi_\theta(a_t^{(i)}|s_t^{(i)})$

Итак, окончательная формула оценки градиента политики:

$\nabla_\theta J(\theta) = \hat{g} = \frac{1}{m} \sum_{i=1}^{m} \sum_{t=0}^{H} \nabla_\theta \log \pi_\theta(a_t^{(i)}|s_t^{(i)}) R(\tau^{(i)})$

6 Практическое занятие

Официальный ноутбук модуля 4 (Google Colab)

Помощь по курсу / Discord

Теперь мы изучили теорию REINFORCE. **Пора реализовать агента на PyTorch** и проверить устойчивость его работы в CartPole-v1 и PixelCopter.

После этого ты сможешь самостоятельно доработать реализацию для более сложных сред.

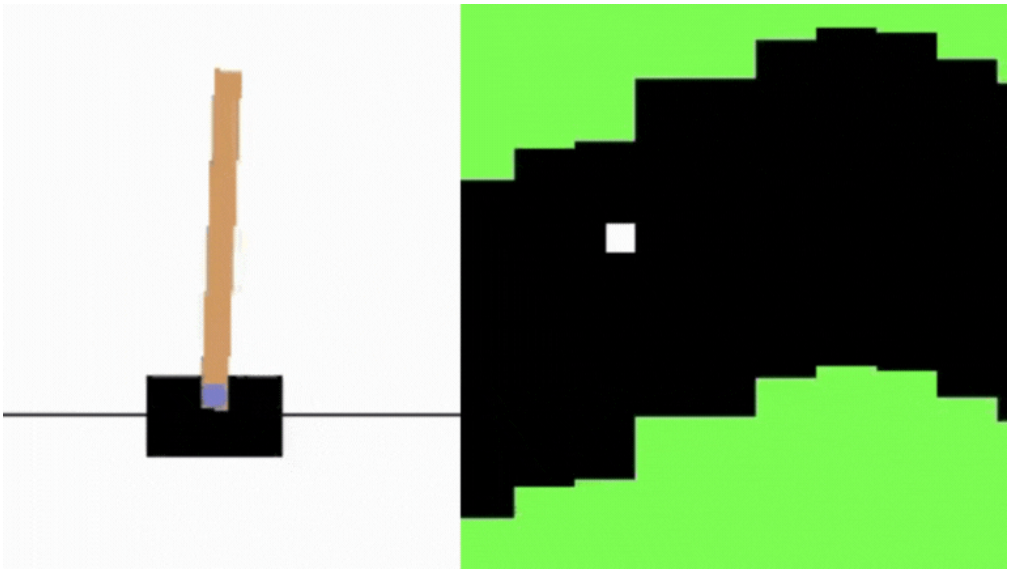


Рис. 23: Среды CartPole и PixelCopter (статичный кадр анимации) [оригинал]

Для зачёта практического задания в системе сертификации необходимо загрузить обученные модели в Hub и:

- Получить результат ≥ 350 в `Cartpole-v1`.
- Получить результат ≥ 5 в `PixelCopter`.

Чтобы узнать результат, открой таблицу лидеров и найди свою модель: **результат = mean_reward - std награды**. **Если модель не отображается, перейди в конец страницы таблицы лидеров и нажми кнопку обновления**.

Если модель не нашлась, прокрути страницу вниз и нажми кнопку обновления.

Подробнее о сертификации читай в соответствующем разделе курса.

Проверить свой прогресс можно здесь: https : / / huggingface . co / spaces / ThomasSimonini / Check-my-progress-Deep-RL-Course.

Чтобы начать практику, нажми кнопку Open In Colab:

Открыть в Colab

Мы настоятельно **рекомендуем студентам выполнять практические задания в Google Colab**, а не на личных компьютерах.

Используя Google Colab, **можно сосредоточиться на обучении и экспериментах, не отвлекаясь на технические детали** настройки окружений.

6.1 Модуль 4: напиши первый алгоритм глубокого RL на PyTorch — REINFORCE — и проверь его устойчивость

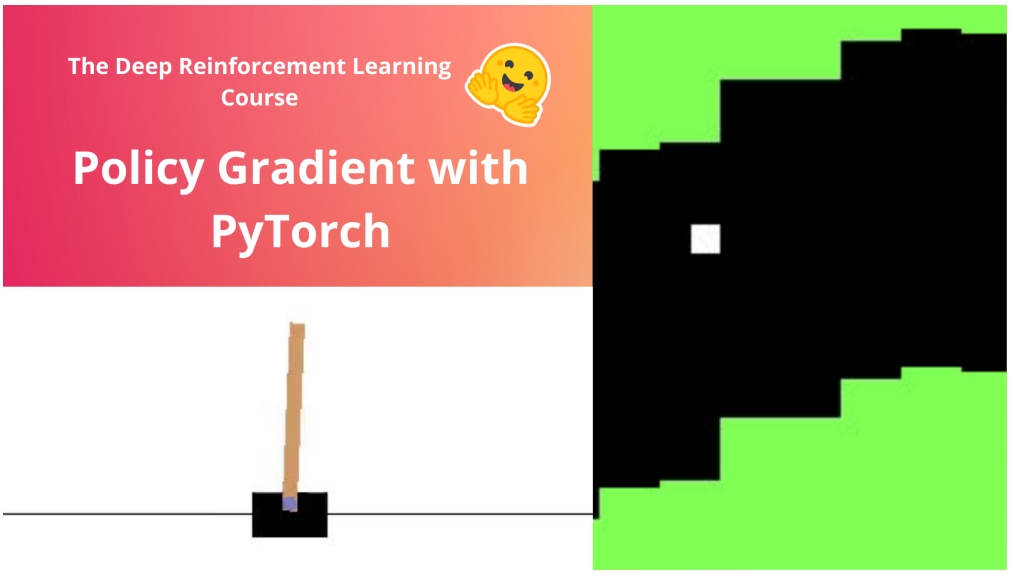


Рис. 24: Обложка модуля 4: градиент политики и REINFORCE [оригинал]

В этом ноутбуке ты с нуля запрограммируешь свой первый алгоритм глубокого обучения с подкреплением — REINFORCE (его также называют градиентом политики Монте-Карло).

REINFORCE относится к *методам на основе политики*: он пытается **непосредственно оптимизировать политику без использования функции ценности действий**.

Точнее, REINFORCE — это *метод градиента политики*, подкласс методов на основе политики. Его цель — **напрямую оптимизировать политику, оценивая веса оптимальной политики с помощью градиентного подъёма**.

Чтобы проверить устойчивость алгоритма, обучим его в двух разных простых средах:

- Cartpole-v1.
- PixelcopterEnv.

Вот пример того, **что ты получишь к концу этого ноутбука.**

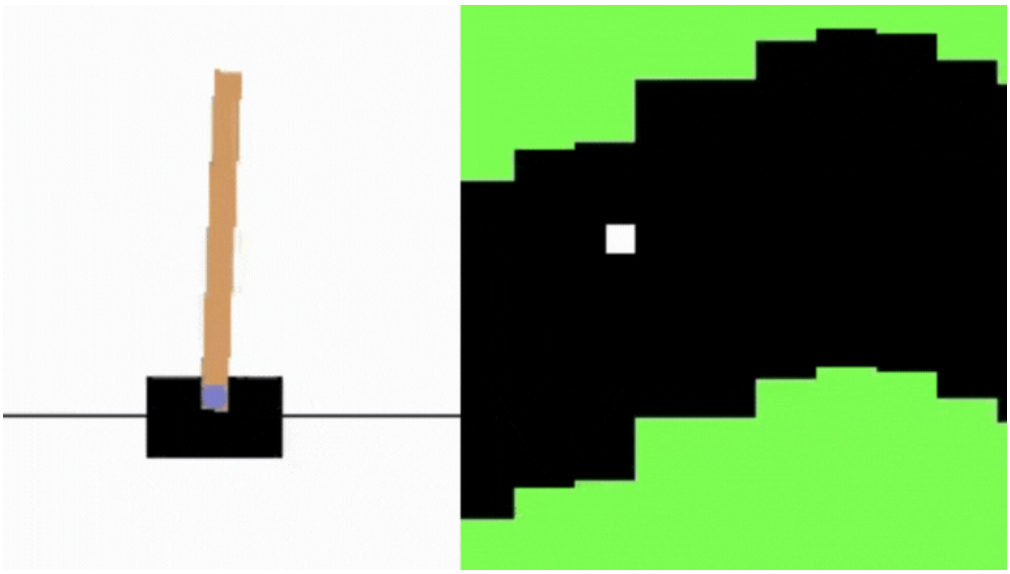


Рис. 25: Среды CartPole и PixelCopter (статичный кадр анимации) [оригинал]

Среды:

- CartPole-v1
- PixelCopter

Библиотеки RL:

- Python
- PyTorch

Мы постоянно улучшаем наши учебные материалы. Поэтому **если заметишь проблемы в этом ноутбуке**, пожалуйста, создай issue в репозитории GitHub.

Цели ноутбука

После выполнения ноутбука ты сможешь:

- **Написать алгоритм REINFORCE с нуля на PyTorch.**
- **Проверить устойчивость агента в простых средах.**
- **Опубликовать обученного агента в Hugging Face Hub** вместе с видеозаписью и результатами оценки.

Предварительные требования

Прежде чем приступать к ноутбуку, необходимо:

Изучить градиенты политики в модуле 4.

6.2 Реализуем REINFORCE с нуля

Несколько советов

Лучше выполнять этот ноутбук в копии на своём Google Диске. Тогда, **если сеанс Colab завершится по тайм-ауту**, сохранённый ноутбук останется на Google Диске и не придётся начинать всё сначала.

Для этого нажми `Ctrl + S` или выбери `File > Save a copy in Google Drive`.

Подключаем GPU

- Чтобы **ускорить обучение агента, используем GPU**. Для этого выбери `Runtime > Change Runtime type`.

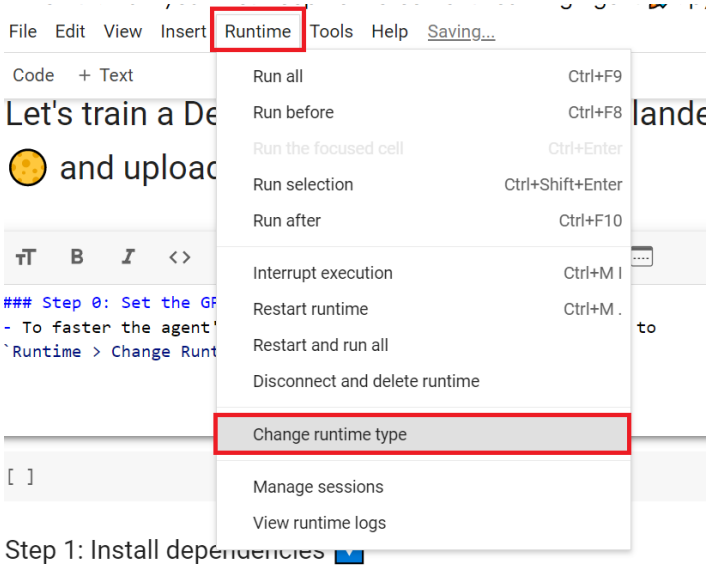


Рис. 26: Выбор режима выполнения Colab: первый шаг [оригинал]

- `Hardware Accelerator > GPU`.

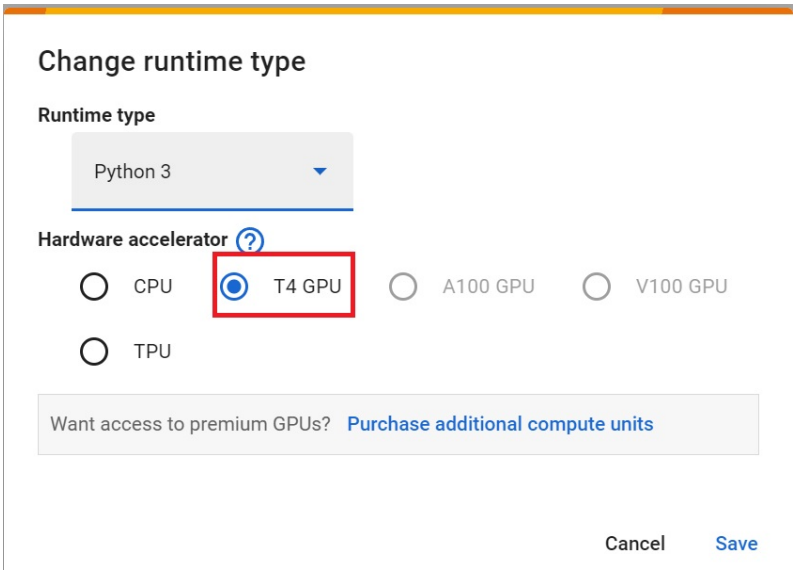


Рис. 27: Выбор аппаратного ускорителя GPU в Colab [оригинал]

Создаём виртуальный дисплей

Во время работы с ноутбуком нам понадобится создавать видеозаписи прохождений. Для этого в Colab **нужен виртуальный экран, позволяющий отображать среду** (и сохранять её кадры).

Следующая ячейка установит библиотеки, создаст виртуальный экран и запустит его.

```
%%capture
!apt install python-opengl
!apt install ffmpeg
!apt install xvfb
!pip install pyvirtualdisplay
!pip install pyglet==1.5.1
```

```
# Virtual display
from pyvirtualdisplay import Display

virtual_display = Display(visible=0, size=(1400, 900))
virtual_display.start()
```

Устанавливаем зависимости

Сначала установим необходимые зависимости. Их несколько:

- `gym`.
- `gym-games`: дополнительные среды Gym, созданные с помощью PyGame.
- `huggingface_hub`: Hub — центральная площадка, на которой можно публиковать и изучать модели и датасеты. Здесь есть версионирование, метрики, визуализации и другие возможности для совместной работы.

Возможно, ты удивляешься, почему мы устанавливаем Gym, а не более новую Gymnasium. **Потому что используемая нами библиотека gym-games ещё не была адаптирована для Gymnasium**.

Основные различия, с которыми ты столкнёшься:

- В `gym` вместо отдельных флагов `terminated` и `truncated` используется единственный флаг `done`.
- В `gym` вызов `env.step()` возвращает `state, reward, done, info`.

Подробнее о различиях Gym и Gymnasium можно узнать здесь: https://gymnasium.farama.org/content/migration-guide/.

Все доступные модели REINFORCE можно посмотреть здесь: https://huggingface.co/models?other=reinforce.

А все модели глубокого обучения с подкреплением — здесь: https://huggingface.co/models?pipeline_tag=reinforcement-learning.

```
!pip install -r https://raw.githubusercontent.com/huggingface/deep-rl-class/main/notebooks/unit4/
    requirements-unit4.txt
```

Импортируем пакеты

Помимо установленных библиотек, мы импортируем:

- `imageio`: библиотеку, которая поможет создать видеозапись прохождения.

```
import numpy as np

from collections import deque

import matplotlib.pyplot as plt
%matplotlib inline

# PyTorch
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.distributions import Categorical
```

```
# Gym
import gym
import gym_pygame

# Hugging Face Hub
from huggingface_hub import notebook_login # To log to our Hugging Face account to be able to
    upload models to the Hub.
import imageio
```

Проверяем наличие GPU

- Проверим, доступен ли GPU.
- Если да, ты увидишь `device:cuda0`.

```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
```

```
print(device)
```

Теперь всё готово для реализации алгоритма REINFORCE.

6.3 Первый агент: играем в CartPole-v1

Создаём среду CartPole и разбираемся в её работе

Среда

Почему мы используем простую среду вроде CartPole-v1? Как объясняется в советах по RL, когда реализуешь агента с нуля, нужно **сначала убедиться, что он работает правильно, и отладить его в простых средах**. Так намного легче искать ошибки, прежде чем переходить к сложным задачам.

> Попробуй получить хотя бы какие-то «признаки жизни» на игрушечных задачах.

> Проверяй реализацию, запуская её во всё более сложных средах (результаты можно сравнивать с RL Zoo). На этом этапе обычно приходится подбирать гиперпараметры.

Среда CartPole-v1

> К тележке, движущейся по рельсу без трения, через неуправляемый шарнир прикреплён шест. В начале шест расположен вертикально. Задача — удерживать его в равновесии, прикладывая к тележке силы влево или вправо.

Итак, начинаем с CartPole-v1. Нужно перемещать тележку влево или вправо **так, чтобы шест не падал и оставался в равновесии**.

Эпизод заканчивается, если:

- Угол отклонения шеста превышает $\pm 12°$.
- Координата тележки выходит за пределы $\pm 2{,}4$.
- Длительность эпизода превышает 500 шагов.

На каждом шаге, пока шест удерживается в равновесии, агент получает награду $+1$.

```
env_id = "CartPole-v1"
# Create the env
env = gym.make(env_id)

# Create the evaluation env
eval_env = gym.make(env_id)

# Get the state space and action space
s_size = env.observation_space.shape[0]
a_size = env.action_space.n
```

```
print("_____OBSERVATION SPACE_____ \n")
print("The State Space is: ", s_size)
print("Sample observation", env.observation_space.sample())  # Get a random observation
```

```
print("\n _____ACTION SPACE_____ \n")
print("The Action Space is: ", a_size)
print("Action Space Sample", env.action_space.sample())  # Take a random action
```

Построим архитектуру REINFORCE

Эта реализация основана на трёх примерах:

- Официальный пример RL от PyTorch.
- REINFORCE от Udacity.
- Улучшение интеграции от Chris1nexus.

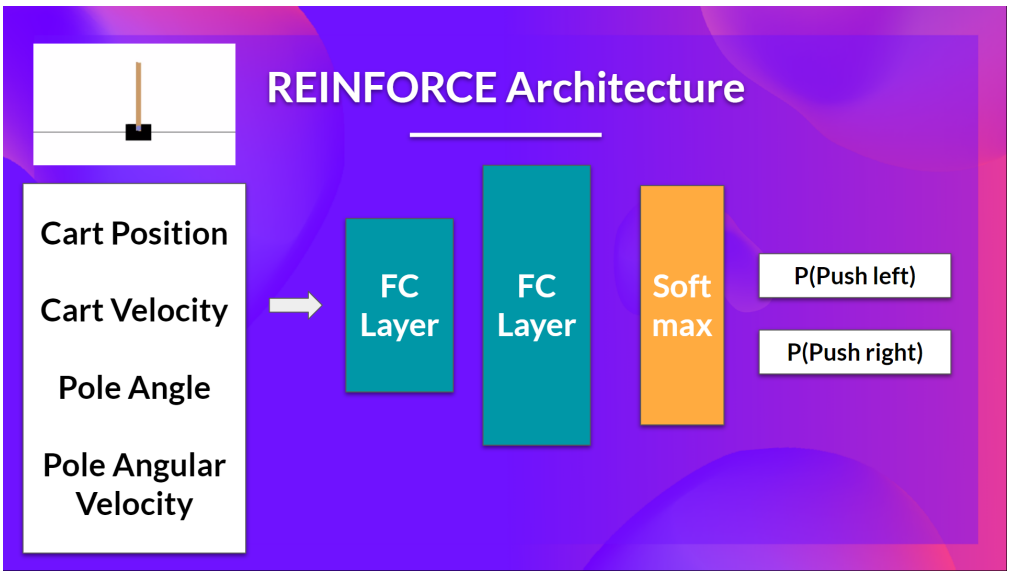


Рис. 28: Архитектура нейросети политики REINFORCE [оригинал]

Итак, нам нужно:

- Два полносвязных слоя (fc1 и fc2).
- Использовать ReLU в качестве функции активации слоя fc1.
- Использовать Softmax, чтобы получить распределение вероятностей действий на выходе.

```
class Policy(nn.Module):
    def __init__(self, s_size, a_size, h_size):
        super(Policy, self).__init__()
```

```
        # Create two fully connected layers



    def forward(self, x):
        # Define the forward pass
        # state goes to fc1 then we apply ReLU activation function

        # fc1 outputs goes to fc2

        # We output the softmax

    def act(self, state):
        """
        Given a state, take action
        """
        state = torch.from_numpy(state).float().unsqueeze(0).to(device)
        probs = self.forward(state).cpu()
        m = Categorical(probs)
        action = np.argmax(m)
        return action.item(), m.log_prob(action)
```

Решение

```
class Policy(nn.Module):
    def __init__(self, s_size, a_size, h_size):
        super(Policy, self).__init__()
        self.fc1 = nn.Linear(s_size, h_size)
        self.fc2 = nn.Linear(h_size, a_size)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return F.softmax(x, dim=1)

    def act(self, state):
        state = torch.from_numpy(state).float().unsqueeze(0).to(device)
        probs = self.forward(state).cpu()
        m = Categorical(probs)
        action = np.argmax(m)
        return action.item(), m.log_prob(action)
```

Я допустил ошибку. Сможешь догадаться, где именно?

- Чтобы выяснить это, выполним прямой проход сети:

```
debug_policy = Policy(s_size, a_size, 64).to(device)
debug_policy.act(env.reset())
```

- Как видно, появляется ошибка `ValueError: The value argument to log_prob must be a Tensor`.
- Она означает, что `action` при вызове `m.log_prob(action)` должен быть тензором (Tensor), **но это не так**.
- Почему так происходит? Посмотри на функцию act и попробуй понять, почему она не работает.

Совет: в этой реализации есть ошибка. Помни, что функция act должна **выбирать случайное действие из распределения вероятностей действий**.

Настоящее решение

```
class Policy(nn.Module):
    def __init__(self, s_size, a_size, h_size):
        super(Policy, self).__init__()
        self.fc1 = nn.Linear(s_size, h_size)
        self.fc2 = nn.Linear(h_size, a_size)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return F.softmax(x, dim=1)

    def act(self, state):
        state = torch.from_numpy(state).float().unsqueeze(0).to(device)
        probs = self.forward(state).cpu()
        m = Categorical(probs)
        action = m.sample()
        return action.item(), m.log_prob(action)
```

Использование CartPole облегчило отладку, поскольку **мы знаем, что ошибка вызвана нашей реализацией, а не простой средой**.

- Поскольку **мы хотим выбирать действие из распределения вероятностей**, нельзя использовать `action = np.argmax(m)`: этот вызов всегда возвращает действие с наибольшей вероятностью.
- Нужно заменить его на `action = m.sample()`, который выбирает действие случайным образом согласно распределению $P(\cdot \mid s)$.

Реализуем алгоритм обучения REINFORCE Псевдокод алгоритма REINFORCE:

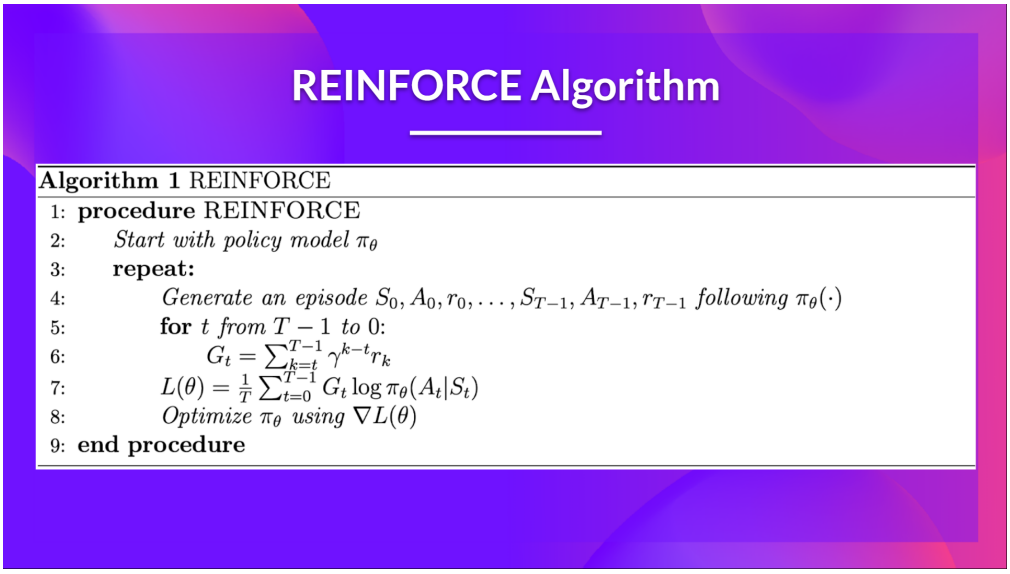


Рис. 29: Псевдокод алгоритма REINFORCE [оригинал]

- При вычислении отдачи G_t (строка 6) видно, что мы суммируем дисконтированные награды, **начиная с момента времени** t.
- Почему? Потому что политика должна **усиливать действия на основании их последствий**: награды, полученные до действия, не имеют значения (они не вызваны этим действием), **важны только последующие награды**.
- Перед реализацией прочитай раздел «Не позволяй прошлому отвлекать тебя», где объясняется использование градиента политики на основе будущих наград (reward-to-go).

Мы используем интересный приём, предложенный Chris1nexus, чтобы **эффективно вычислять отдачу на каждом временном шаге**. Процедура объяснена в комментариях к коду. Также можно прочитать пояснения к PR. Главная идея — **вычислять отдачу для каждого шага без лишних затрат**.

Второй вопрос, который может возникнуть: **почему мы минимизируем функцию потерь**? Разве ранее мы не говорили о градиентном подъёме, а не о спуске?

- Нам нужно максимизировать функцию полезности $J(\theta)$, но в PyTorch и TensorFlow удобнее **минимизировать целевую функцию**.
  - Допустим, на некотором шаге мы хотим усилить действие 3. До обучения его вероятность равна 0,25.
  - Нужно изменить θ так, чтобы $\pi_\theta(a_3|s;\theta) > 0.25$.
  - Поскольку сумма всех вероятностей равна 1, максимизация $\pi_\theta(a_3|s;\theta)$ будет **уменьшать вероятности других действий**.
  - Поэтому нужно указать PyTorch **минимизировать** $1 - \pi_\theta(a_3|s;\theta)$.
  - Такая функция потерь стремится к нулю при приближении $\pi_\theta(a_3|s;\theta)$ к 1.
  - Тем самым мы направляем градиент на увеличение $\pi_\theta(a_3|s;\theta)$.

```
def reinforce(policy, optimizer, n_training_episodes, max_t, gamma, print_every):
    # Help us to calculate the score during the training
    scores_deque = deque(maxlen=100)
    scores = []
    # Line 3 of pseudocode
    for i_episode in range(1, n_training_episodes+1):
        saved_log_probs = []
        rewards = []
        state = # TODO: reset the environment
        # Line 4 of pseudocode
        for t in range(max_t):
            action, log_prob = # TODO get the action
            saved_log_probs.append(log_prob)
            state, reward, done, _ = # TODO: take an env step
            rewards.append(reward)
            if done:
                break
        scores_deque.append(sum(rewards))
        scores.append(sum(rewards))

        # Line 6 of pseudocode: calculate the return
        returns = deque(maxlen=max_t)
        n_steps = len(rewards)
        # Compute the discounted returns at each timestep,
        # as the sum of the gamma-discounted return at time t (G_t) + the reward at time t

        # In O(N) time, where N is the number of time steps
        # (this definition of the discounted return G_t follows the definition of this quantity
        # shown at page 44 of Sutton&Barto 2017 2nd draft)
        # G_t = r_(t+1) + r_(t+2) + ...

        # Given this formulation, the returns at each timestep t can be computed
        # by re-using the computed future returns G_(t+1) to compute the current return G_t
        # G_t = r_(t+1) + gamma*G_(t+1)
        # G_(t-1) = r_t + gamma* G_t
        # (this follows a dynamic programming approach, with which we memorize solutions in order
        # to avoid computing them multiple times)

        # This is correct since the above is equivalent to (see also page 46 of Sutton&Barto 2017 2
    nd draft)
        # G_(t-1) = r_t + gamma*r_(t+1) + gamma*gamma*r_(t+2) + ...
```

```
        ## Given the above, we calculate the returns at timestep t as:
        #               gamma[t] * return[t] + reward[t]
        #
        ## We compute this starting from the last timestep to the first, in order
        ## to employ the formula presented above and avoid redundant computations that would be
    needed
        ## if we were to do it from first to last.

        ## Hence, the queue "returns" will hold the returns in chronological order, from t=0 to t=
    n_steps
        ## thanks to the appendleft() function which allows to append to the position 0 in constant
     time O(1)
        ## a normal python list would instead require O(N) to do this.
        for t in range(n_steps)[::-1]:
            disc_return_t = (returns[0] if len(returns)>0 else 0)
            returns.appendleft(    ) # TODO: complete here

        ## standardization of the returns is employed to make training more stable
        eps = np.finfo(np.float32).eps.item()

        ## eps is the smallest representable float, which is
        # added to the standard deviation of the returns to avoid numerical instabilities
        returns = torch.tensor(returns)
        returns = (returns - returns.mean()) / (returns.std() + eps)

        # Line 7:
        policy_loss = []
        for log_prob, disc_return in zip(saved_log_probs, returns):
            policy_loss.append(-log_prob * disc_return)
        policy_loss = torch.cat(policy_loss).sum()

        # Line 8: PyTorch prefers gradient descent
        optimizer.zero_grad()
        policy_loss.backward()
        optimizer.step()

        if i_episode % print_every == 0:
            print('Episode {}\tAverage Score: {:.2f}'.format(i_episode, np.mean(scores_deque)))

    return scores
```

Решение

```
def reinforce(policy, optimizer, n_training_episodes, max_t, gamma, print_every):
    # Help us to calculate the score during the training
    scores_deque = deque(maxlen=100)
    scores = []
    # Line 3 of pseudocode
    for i_episode in range(1, n_training_episodes + 1):
        saved_log_probs = []
        rewards = []
        state = env.reset()
        # Line 4 of pseudocode
        for t in range(max_t):
            action, log_prob = policy.act(state)
            saved_log_probs.append(log_prob)
            state, reward, done, _ = env.step(action)
            rewards.append(reward)
            if done:
                break
        scores_deque.append(sum(rewards))
        scores.append(sum(rewards))
```

```
        # Line 6 of pseudocode: calculate the return
        returns = deque(maxlen=max_t)
        n_steps = len(rewards)
        # Compute the discounted returns at each timestep,
        # as
        #      the sum of the gamma-discounted return at time t (G_t) + the reward at time t
        #
        # In O(N) time, where N is the number of time steps
        # (this definition of the discounted return G_t follows the definition of this quantity
        # shown at page 44 of Sutton&Barto 2017 2nd draft)
        # G_t = r_(t+1) + r_(t+2) + ...

        # Given this formulation, the returns at each timestep t can be computed
        # by re-using the computed future returns G_(t+1) to compute the current return G_t
        # G_t = r_(t+1) + gamma*G_(t+1)
        # G_(t-1) = r_t + gamma* G_t
        # (this follows a dynamic programming approach, with which we memorize solutions in order
        # to avoid computing them multiple times)

        # This is correct since the above is equivalent to (see also page 46 of Sutton&Barto 2017 2
    nd draft)
        # G_(t-1) = r_t + gamma*r_(t+1) + gamma*gamma*r_(t+2) + ...

        ## Given the above, we calculate the returns at timestep t as:
        #               gamma[t] * return[t] + reward[t]
        #
        ## We compute this starting from the last timestep to the first, in order
        ## to employ the formula presented above and avoid redundant computations that would be
    needed
        ## if we were to do it from first to last.

        ## Hence, the queue "returns" will hold the returns in chronological order, from t=0 to t=
    n_steps
        ## thanks to the appendleft() function which allows to append to the position 0 in constant
     time O(1)
        ## a normal python list would instead require O(N) to do this.
        for t in range(n_steps)[::-1]:
            disc_return_t = returns[0] if len(returns) > 0 else 0
            returns.appendleft(gamma * disc_return_t + rewards[t])

        ## standardization of the returns is employed to make training more stable
        eps = np.finfo(np.float32).eps.item()
        ## eps is the smallest representable float, which is
        # added to the standard deviation of the returns to avoid numerical instabilities
        returns = torch.tensor(returns)
        returns = (returns - returns.mean()) / (returns.std() + eps)

        # Line 7:
        policy_loss = []
        for log_prob, disc_return in zip(saved_log_probs, returns):
            policy_loss.append(-log_prob * disc_return)
        policy_loss = torch.cat(policy_loss).sum()

        # Line 8: PyTorch prefers gradient descent
        optimizer.zero_grad()
        policy_loss.backward()
        optimizer.step()

        if i_episode % print_every == 0:
            print("Episode {}\tAverage Score: {:.2f}".format(i_episode, np.mean(scores_deque)))

    return scores
```

Обучаем агента

- Теперь всё готово к обучению агента.
- Но сначала зададим переменную, содержащую все гиперпараметры обучения.
- Ты можешь (и даже должен) изменять параметры обучения.

```
cartpole_hyperparameters = {
    "h_size": 16,
    "n_training_episodes": 1000,
    "n_evaluation_episodes": 10,
    "max_t": 1000,
    "gamma": 1.0,
    "lr": 1e-2,
    "env_id": env_id,
    "state_space": s_size,
    "action_space": a_size,
}
```

```
# Create policy and place it to the device
cartpole_policy = Policy(
    cartpole_hyperparameters["state_space"],
    cartpole_hyperparameters["action_space"],
    cartpole_hyperparameters["h_size"],
).to(device)
cartpole_optimizer = optim.Adam(cartpole_policy.parameters(), lr=cartpole_hyperparameters["lr"])
```

```
scores = reinforce(
    cartpole_policy,
    cartpole_optimizer,
    cartpole_hyperparameters["n_training_episodes"],
    cartpole_hyperparameters["max_t"],
    cartpole_hyperparameters["gamma"],
    100,
)
```

Определяем метод оценки

- Здесь мы определим метод оценки, с помощью которого будем тестировать нашего агента REINFORCE.

```
def evaluate_agent(env, max_steps, n_eval_episodes, policy):
    """
    Evaluate the agent for ``n_eval_episodes`` episodes and returns average reward and std of
    reward.
    :param env: The evaluation environment
    :param n_eval_episodes: Number of episode to evaluate the agent
    :param policy: The Reinforce agent
    """
    episode_rewards = []
    for episode in range(n_eval_episodes):
        state = env.reset()
        step = 0
        done = False
        total_rewards_ep = 0

        for step in range(max_steps):
            action, _ = policy.act(state)
            new_state, reward, done, info = env.step(action)
            total_rewards_ep += reward
```

```
            if done:
                break
            state = new_state
        episode_rewards.append(total_rewards_ep)
    mean_reward = np.mean(episode_rewards)
    std_reward = np.std(episode_rewards)

    return mean_reward, std_reward
```

Оцениваем работу агента

```
evaluate_agent(
    eval_env, cartpole_hyperparameters["max_t"], cartpole_hyperparameters["n_evaluation_episodes"],
     cartpole_policy
)
```

Публикуем обученную модель на Hub Теперь, убедившись, что обучение дало хорошие результаты, мы можем опубликовать обученную модель на Hub одной строкой кода.

Вот пример карточки модели (Model Card):

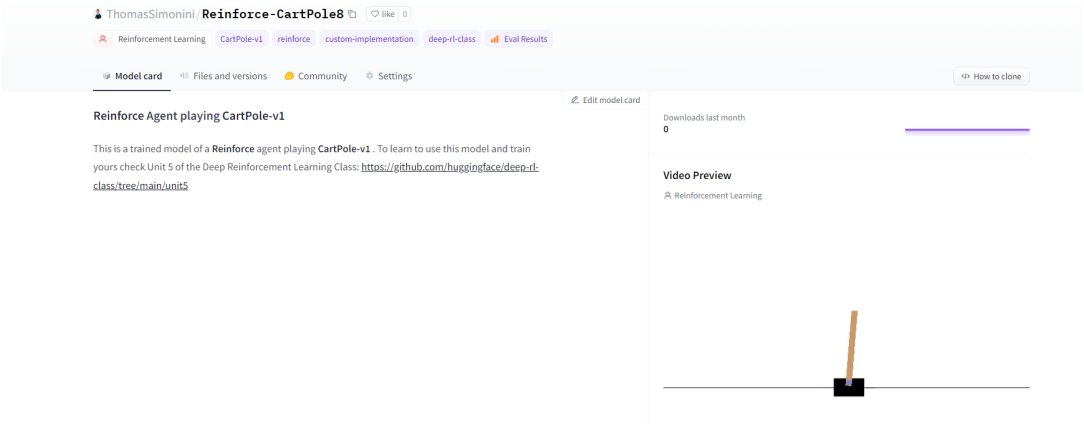


Рис. 30: Интерфейс карточки модели в Hugging Face [оригинал]

Загружаем модель на Hub

Не изменяй этот код

```
from huggingface_hub import HfApi, snapshot_download
from huggingface_hub.repocard import metadata_eval_result, metadata_save

from pathlib import Path
import datetime
import json
import imageio

import tempfile

import os
```

```
def record_video(env, policy, out_directory, fps=30):
    """
    Generate a replay video of the agent
    :param env
```

```
    :param Qtable: Qtable of our agent
    :param out_directory
    :param fps: how many frame per seconds (with taxi-v3 and frozenlake-v1 we use 1)
    """
    images = []
    done = False
    state = env.reset()
    img = env.render(mode="rgb_array")
    images.append(img)
    while not done:
        # Take the action (index) that have the maximum expected future reward given that state
        action, _ = policy.act(state)
        state, reward, done, info = env.step(action)  # We directly put next_state = state for
    recording logic
        img = env.render(mode="rgb_array")
        images.append(img)
    imageio.mimsave(out_directory, [np.array(img) for i, img in enumerate(images)], fps=fps)
```

```
def push_to_hub(repo_id,
                model,
                hyperparameters,
                eval_env,
                video_fps=30
                ):
  """
  Evaluate, Generate a video and Upload a model to Hugging Face Hub.
  This method does the complete pipeline:
  - It evaluates the model
  - It generates the model card
  - It generates a replay video of the agent
  - It pushes everything to the Hub

  :param repo_id: repo_id: id of the model repository from the Hugging Face Hub
  :param model: the pytorch model we want to save
  :param hyperparameters: training hyperparameters
  :param eval_env: evaluation environment
  :param video_fps: how many frame per seconds to record our video replay
  """

  _, repo_name = repo_id.split("/")
  api = HfApi()

  # Step 1: Create the repo
  repo_url = api.create_repo(
        repo_id=repo_id,
        exist_ok=True,
  )

  with tempfile.TemporaryDirectory() as tmpdirname:
    local_directory = Path(tmpdirname)

    # Step 2: Save the model
    torch.save(model, local_directory / "model.pt")

    # Step 3: Save the hyperparameters to JSON
    with open(local_directory / "hyperparameters.json", "w") as outfile:
      json.dump(hyperparameters, outfile)

    # Step 4: Evaluate the model and build JSON
    mean_reward, std_reward = evaluate_agent(eval_env,
                                             hyperparameters["max_t"],
                                             hyperparameters["n_evaluation_episodes"],
                                             model)
    # Get datetime
```

```
  eval_datetime = datetime.datetime.now()
  eval_form_datetime = eval_datetime.isoformat()

  evaluate_data = {
        "env_id": hyperparameters["env_id"],
        "mean_reward": mean_reward,
        "n_evaluation_episodes": hyperparameters["n_evaluation_episodes"],
        "eval_datetime": eval_form_datetime,
  }

  # Write a JSON file
  with open(local_directory / "results.json", "w") as outfile:
      json.dump(evaluate_data, outfile)

  # Step 5: Create the model card
  env_name = hyperparameters["env_id"]

  metadata = {}
  metadata["tags"] = [
        env_name,
        "reinforce",
        "reinforcement-learning",
        "custom-implementation",
        "deep-rl-class"
    ]

  # Add metrics
  eval = metadata_eval_result(
      model_pretty_name=repo_name,
      task_pretty_name="reinforcement-learning",
      task_id="reinforcement-learning",
      metrics_pretty_name="mean_reward",
      metrics_id="mean_reward",
      metrics_value=f"{mean_reward:.2f} +/- {std_reward:.2f}",
      dataset_pretty_name=env_name,
      dataset_id=env_name,
    )

  # Merges both dictionaries
  metadata = {**metadata, **eval}

  model_card = f"""
# **Reinforce** Agent playing **{env_id}**
This is a trained model of a **Reinforce** agent playing **{env_id}** .
To learn to use this model and train yours check Unit 4 of the Deep Reinforcement Learning Course:
   https://huggingface.co/deep-rl-course/unit4/introduction
"""

  readme_path = local_directory / "README.md"
  readme = ""
  if readme_path.exists():
      with readme_path.open("r", encoding="utf8") as f:
        readme = f.read()
  else:
    readme = model_card

  with readme_path.open("w", encoding="utf-8") as f:
    f.write(readme)

  # Save our metrics to Readme metadata
  metadata_save(readme_path, metadata)

  # Step 6: Record a video
  video_path =  local_directory / "replay.mp4"
  record_video(env, model, video_path, video_fps)
```

```
# Step 7. Push everything to the Hub
api.upload_folder(
      repo_id=repo_id,
      folder_path=local_directory,
      path_in_repo=".",
)

print(f"Your model is pushed to the Hub. You can view your model here: {repo_url}")
```

С помощью `push_to_hub` **ты можешь оценить агента, записать видео его работы, создать карточку модели и загрузить её на Hub**.

Таким образом:

- Ты сможешь **показать результаты своей работы**.
- Ты сможешь **посмотреть, как играет твой агент**.
- Ты сможешь **поделиться агентом с сообществом, чтобы другие тоже могли его использовать**.
- Ты сможешь **открыть таблицу лидеров и сравнить качество работы своего агента с результатами однокурсников**: https://huggingface.co/spaces/huggingface-projects/Deep-Reinforcement-Learning-Leaderboard.

Чтобы поделиться моделью с сообществом, осталось выполнить ещё три шага:

1. Если учётной записи Hugging Face пока нет, создай её: https://huggingface.co/join.
2. Войди в аккаунт, затем сохрани токен аутентификации с сайта Hugging Face.

- Создай новый токен на странице https://huggingface.co/settings/tokens **с правами записи (write)**.

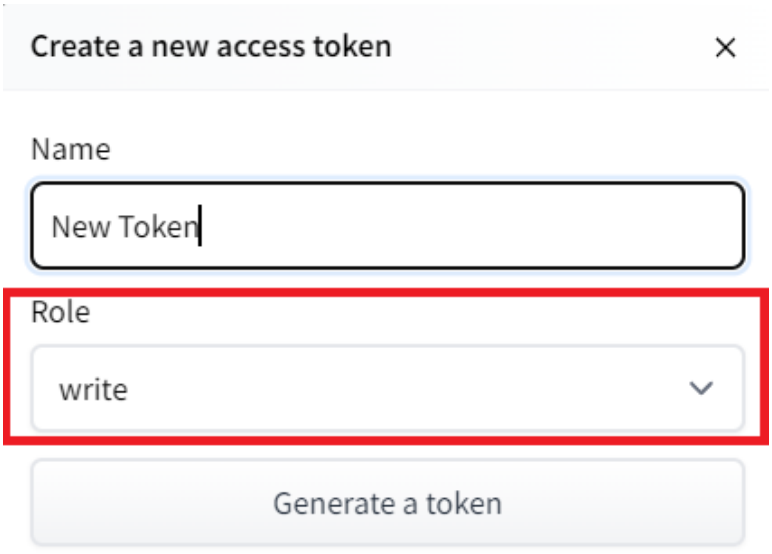


Рис. 31: Создание токена доступа Hugging Face [оригинал]

```
notebook_login()
```

Если ты не используешь Google Colab или Jupyter Notebook, вместо этого выполни команду `huggingface-cli login` (или `login`).

3. Теперь мы готовы отправить обученного агента в Hub. Для этого используем функцию `package_to_hub()`.

```
repo_id = ""  # TODO Define your repo id {username/Reinforce-{model-id}}
push_to_hub(
    repo_id,
    cartpole_policy,  # The model we want to save
    cartpole_hyperparameters,  # Hyperparameters
    eval_env,  # Evaluation environment
    video_fps=30
)
```

Проверив устойчивость нашей реализации, перейдём к более сложной среде — PixelCopter.

Второй агент: PixelCopter

Изучаем среду PixelCopter

- Документация среды.

```
env_id = "Pixelcopter-PLE-v0"
env = gym.make(env_id)
eval_env = gym.make(env_id)
s_size = env.observation_space.shape[0]
a_size = env.action_space.n
```

```
print("_____OBSERVATION SPACE_____ \n")
print("The State Space is: ", s_size)
print("Sample observation", env.observation_space.sample())  # Get a random observation
```

```
print("\n _____ACTION SPACE_____ \n")
print("The Action Space is: ", a_size)
print("Action Space Sample", env.action_space.sample())  # Take a random action
```

Пространство наблюдений (7 компонентов):

- Координата игрока по оси y.
- Скорость игрока.
- Расстояние от игрока до пола.
- Расстояние от игрока до потолка.
- Расстояние по оси x от игрока до следующего препятствия.
- Координата верхней границы следующего препятствия по оси y.
- Координата нижней границы следующего препятствия по оси y.

Пространство действий (2 действия):

- Подняться вверх (нажать ускоритель).
- Ничего не делать (не нажимать ускоритель).

Функция вознаграждения:

- За прохождение каждого вертикального препятствия агент получает положительную награду +1. При достижении терминального состояния он получает отрицательную награду -1.

Определяем новую политику

- Нам нужна более глубокая нейронная сеть, поскольку среда стала сложнее.

```
class Policy(nn.Module):
    def __init__(self, s_size, a_size, h_size):
        super(Policy, self).__init__()
        # Define the three layers here

    def forward(self, x):
        # Define the forward process here
        return F.softmax(x, dim=1)

    def act(self, state):
        state = torch.from_numpy(state).float().unsqueeze(0).to(device)
        probs = self.forward(state).cpu()
        m = Categorical(probs)
        action = m.sample()
        return action.item(), m.log_prob(action)
```

Решение

```
class Policy(nn.Module):
    def __init__(self, s_size, a_size, h_size):
        super(Policy, self).__init__()
        self.fc1 = nn.Linear(s_size, h_size)
        self.fc2 = nn.Linear(h_size, h_size * 2)
        self.fc3 = nn.Linear(h_size * 2, a_size)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return F.softmax(x, dim=1)

    def act(self, state):
        state = torch.from_numpy(state).float().unsqueeze(0).to(device)
        probs = self.forward(state).cpu()
        m = Categorical(probs)
        action = m.sample()
        return action.item(), m.log_prob(action)
```

Задаём гиперпараметры

- Потому что эта среда сложнее.
- Особенно важно увеличить число нейронов в скрытых слоях.

```
pixelcopter_hyperparameters = {
    "h_size": 64,
    "n_training_episodes": 50000,
    "n_evaluation_episodes": 10,
    "max_t": 10000,
    "gamma": 0.99,
    "lr": 1e-4,
    "env_id": env_id,
    "state_space": s_size,
    "action_space": a_size,
}
```

Обучаем агента

- Теперь всё готово для обучения агента.

```
# Create policy and place it to the device
# torch.manual_seed(50)
pixelcopter_policy = Policy(
    pixelcopter_hyperparameters["state_space"],
    pixelcopter_hyperparameters["action_space"],
    pixelcopter_hyperparameters["h_size"],
).to(device)
pixelcopter_optimizer = optim.Adam(pixelcopter_policy.parameters(), lr=pixelcopter_hyperparameters
    ["lr"])
```

```
scores = reinforce(
    pixelcopter_policy,
    pixelcopter_optimizer,
    pixelcopter_hyperparameters["n_training_episodes"],
    pixelcopter_hyperparameters["max_t"],
    pixelcopter_hyperparameters["gamma"],
    1000,
)
```

Публикуем обученную модель на Hub

```
repo_id = ""  # TODO Define your repo id {username/Reinforce-{model-id}}
push_to_hub(
    repo_id,
    pixelcopter_policy,  # The model we want to save
    pixelcopter_hyperparameters,  # Hyperparameters
    eval_env,  # Evaluation environment
    video_fps=30
)
```

Дополнительные задания

Лучший способ научиться — **экспериментировать самостоятельно**! Как ты видел, сейчас агент играет не слишком хорошо. Для начала можно увеличить число шагов обучения. Но стоит также подобрать лучшие параметры.

В таблице лидеров можно найти своих агентов. Сможешь подняться на первое место?

Вот несколько идей, которые помогут улучшить позицию в таблице лидеров:

- Обучай агент дольше.
- Пробуй разные гиперпараметры, изучая эксперименты других участников: https : / / huggingface.co/models?other=reinforce.
- **Загружай на Hub новые версии обученных моделей**.
- **Улучши реализацию для более сложных сред**. Например, что будет, если заменить сеть на свёрточную нейронную сеть (CNN) для обработки изображений кадров в качестве наблюдений?

Поздравляем с завершением модуля! Материала было немало. И поздравляем с окончанием практики: ты только что с нуля написал своего первого агента глубокого RL на PyTorch и опубликовал его на Hub.

Не бойся продолжать эксперименты с этим модулем, **улучшая реализацию для более сложных сред**. Например, можно попробовать свёрточную нейронную сеть для обработки кадров-наблюдений.

В следующем модуле мы **подробнее познакомимся с Unity ML-Agents**, обучая агентов в средах Unity. Это подготовит тебя к **соревнованиям AI vs AI, где агенты будут противостоять друг другу в снежной битве и футбольном матче**.

Звучит интересно? До встречи в следующем модуле!

Наконец, нам очень хотелось бы **услышать твоё мнение о курсе и о том, как его можно улучшить**. Если есть предложения, пожалуйста, заполни эту форму.

До встречи в модуле 5!

Продолжай учиться и оставайся молодцом

7 Тест

Лучший способ учиться и не поддаваться иллюзии компетентности — **проверять себя**. Так можно понять, **какие знания нужно закрепить**.

Вопрос 1. Каковы преимущества градиента политики перед методами на основе ценности? (Выбери все подходящие варианты)

- **A.** Методы градиента политики могут обучать стохастическую политику.
- **B.** Методы градиента политики эффективнее при высокой размерности пространства действий и в пространствах непрерывных действий.
- **C.** Методы градиента политики чаще всего сходятся к глобальному максимуму.

Вопрос 2. Что такое теорема о градиенте политики?

Вопрос 3. Чем отличаются методы на основе политики от методов градиента политики? (Выбери все подходящие варианты)

- **A.** Методы на основе политики являются подмножеством методов градиента политики.
- **B.** Методы градиента политики являются подмножеством методов на основе политики.
- **C.** В методах на основе политики параметры θ можно оптимизировать **косвенно**, максимизируя локальную аппроксимацию целевой функции методами восхождения на холм, имитации отжига или эволюционных стратегий.
- **D.** В методах градиента политики параметры θ оптимизируются **напрямую** с помощью градиентного подъёма целевой функции.

Вопрос 4. Почему для оптимизации $J(\theta)$ используется градиентный подъём, а не спуск?

- **A.** Мы хотим минимизировать $J(\theta)$, а градиентный подъём задаёт направление наиболее быстрого увеличения $J(\theta)$.
- **B.** Мы хотим максимизировать $J(\theta)$, а градиентный подъём задаёт направление наиболее быстрого увеличения $J(\theta)$.

Поздравляем с завершением теста! Если в каких-то вопросах ошибся, перечитай соответствующие разделы, чтобы закрепить материал.

7.1 Ответы на тест и пояснения

Ответ на вопрос 1

Правильные ответы: A, B.

A. Методы градиента политики могут обучать стохастическую политику.

B. Методы градиента политики эффективнее при высокой размерности пространства действий и в пространствах непрерывных действий.

C. Методы градиента политики чаще всего сходятся к глобальному максимуму. Пояснение: нет, часто градиент политики сходится к локальному максимуму, а не к глобальному оптимуму.

Ответ на вопрос 2

Теорема о градиенте политики позволяет преобразовать целевую функцию в дифференцируемое выражение, не требующее дифференцирования распределения состояний.

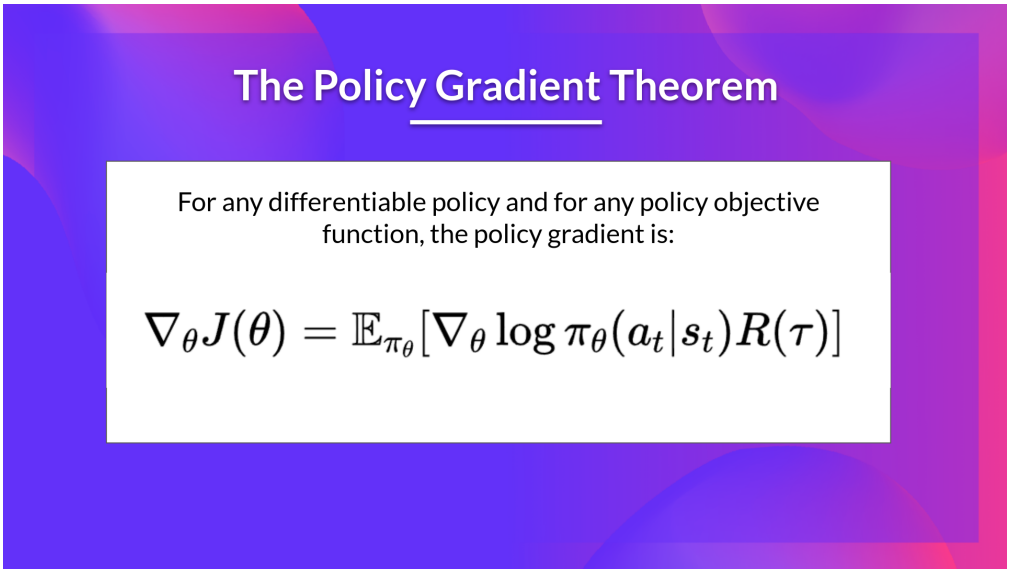


Рис. 32: Теорема о градиенте политики [оригинал]

Ответ на вопрос 3

Правильные ответы: B, C, D.

A. Методы на основе политики являются подмножеством методов градиента политики.

B. Методы градиента политики являются подмножеством методов на основе политики.

C. В методах на основе политики параметры θ можно оптимизировать **косвенно**, максимизируя локальную аппроксимацию целевой функции методами восхождения на холм, имитации отжига или эволюционных стратегий.

D. В методах градиента политики параметры θ оптимизируются **напрямую** с помощью градиентного подъёма целевой функции.

Ответ на вопрос 4

Правильный ответ: B.

A. Мы хотим минимизировать $J(\theta)$, а градиентный подъём задаёт направление наиболее быстрого увеличения $J(\theta)$.

B. Мы хотим максимизировать $J(\theta)$, а градиентный подъём задаёт направление наиболее быстрого увеличения $J(\theta)$.

8 Заключение

Поздравляем с завершением модуля! Материала было много. И поздравляем с окончанием практики: ты с нуля реализовал своего первого агента глубокого обучения с подкреплением на PyTorch и опубликовал его на Hub.

Продолжай экспериментировать с этим модулем, **улучшая реализацию для более сложных сред**. Например, почему бы не заменить сеть свёрточной нейронной сетью для обработки кадров-наблюдений?

В следующем модуле мы **подробнее познакомимся с Unity ML-Agents** и обучим агентов в средах Unity. Благодаря этому ты будешь готов к **соревнованиям AI vs AI, где агенты противостоят друг другу в снежной битве и футбольном матче**.

Звучит интересно? До встречи в следующий раз!

Наконец, нам очень хотелось бы **услышать твоё мнение о курсе и о том, как его можно улучшить**. Если есть предложения, пожалуйста, заполни эту форму.

Продолжай учиться и оставайся молодцом

9 Глоссарий

Этот глоссарий создан сообществом. Мы будем рады твоему вкладу!

- **Глубокое Q-обучение (Deep Q-Learning):** алгоритм глубокого обучения с подкреплением на основе ценности, использующий глубокую нейронную сеть для аппроксимации Q-значений действий в данном состоянии. Цель — найти оптимальную политику, максимизирующую ожидаемую суммарную награду, путём обучения ценностей действий.
- **Методы на основе ценности (value-based):** методы обучения с подкреплением, в которых функция ценности оценивается как промежуточный этап поиска оптимальной политики.
- **Методы на основе политики (policy-based):** методы обучения с подкреплением, которые непосредственно аппроксимируют оптимальную политику без обучения функции ценности. На практике они выдают распределение вероятностей действий.

  Преимущества методов градиента политики перед методами на основе ценности включают:

  - Простоту интеграции: не нужно хранить значения ценности действий.
  - Возможность обучать стохастическую политику: агент исследует пространство состояний, не повторяя одну и ту же траекторию, и избегает проблемы неразличимости состояний по наблюдениям.
  - Эффективность при большой размерности пространства действий и в непрерывных пространствах действий.
  - Более благоприятные свойства сходимости.

- **Градиент политики (Policy Gradient):** подкласс методов на основе политики, где целью является максимизация качества параметризованной политики с помощью градиентного подъёма. Смысл градиента политики — управлять распределением вероятностей действий, настраивая политику так, чтобы хорошие действия (повышающие отдачу) выбирались чаще в будущем.
- **Monte Carlo REINFORCE:** алгоритм градиента политики, который использует оценку отдачи за весь эпизод для обновления параметров политики.

Если хочешь помочь улучшить курс, можешь создать Pull Request.

Создание этого глоссария стало возможным благодаря следующим участникам:

- Diego Carpintero

10 Дополнительная литература

Эти материалы **необязательны**: они пригодятся, если захочешь изучить тему глубже.

10.1 Введение в оптимизацию политики

- Часть 3. Введение в оптимизацию политики — документация Spinning Up.

10.2 Градиент политики

- https://johnwlambert.github.io/policy-gradients/
- Объяснение градиентов политики в RL.
- Глава 13 «Методы градиента политики» в книге Reinforcement Learning: An Introduction Ричарда Саттона и Эндрю Барто.

10.3 Реализация

- Реализация REINFORCE на PyTorch.
- Реализации алгоритмов от DDPG до PPO.